

MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

Interactive Machine Learning Curriculum

Theory, Hands-on Exercises & Complete Reference Solutions

A comprehensive, self-paced curriculum spanning 20 interactive chapters. Covers foundational data manipulation, preprocessing, and plotting, classical regression and classification algorithms, unsupervised clustering, feature engineering, time-series forecasting, NLP, recommender systems, and deep learning.

TEXTBOOK

Format: Comprehensive Textbook & Workbook

Chapters: 20 Curriculum **Exercises:** 60 Interactive Challenges (Easy, Medium, Hard)

Table of Contents

Chapter 1: Introduction & Data Manipulation (Pandas & NumPy)	3
Chapter 2: Data Cleaning & Preprocessing	8
Chapter 3: Data Visualization & Plotting	13
Chapter 4: Regression (Linear & Logistic)	17
Chapter 5: Decision Trees & Random Forests	22
Chapter 6: Support Vector Machines & Naive Bayes	27
Chapter 7: Unsupervised Learning & Clustering	32
Chapter 8: Feature Engineering & Selection	36
Chapter 9: Model Evaluation & Hyperparameter Tuning	40
Chapter 10: Introduction to Deep Learning	44
Chapter 11: Time Series Analysis & Forecasting	49
Chapter 12: Natural Language Processing (NLP)	54
Chapter 13: Anomaly Detection	59
Chapter 14: Ensemble Learning & Gradient Boosting	64
Chapter 15: Dimensionality Reduction & Manifold Learning	69
Chapter 16: Recommender Systems	74
Chapter 17: Association Rule Learning	79
Chapter 18: Semi-Supervised Learning	84
Chapter 19: Classification on Imbalanced Data	89
Chapter 20: Hyperparameter Optimization with Optuna	94
Appendix: Exercise Solutions	99

CHAPTER 1

Introduction & Data Manipulation (Pandas & NumPy)

Theory & Foundations

In this notebook, you will explore the building blocks of data science and machine learning in Python: **NumPy** (Numerical Python) and **Pandas** (Python Data Analysis Library). Almost every machine learning pipeline starts with loading, manipulating, and understanding raw data.

1. NumPy: Vectorization and Arrays

NumPy provides the `ndarray` object, a multidimensional array that is memory-efficient and supports fast vector mathematical operations. - **Vectorization**: Instead of writing nested `for` loops in Python, we perform operations directly on arrays. - **Slicing and Indexing**: Just like Python lists, but extends to multiple dimensions.

2. Pandas: DataFrames and Series

Pandas is built on top of NumPy and provides: - **Series**: A 1D array with labeled axes. - **DataFrame**: A 2D labeled data structure with columns of potentially different types (similar to a SQL table). - **Data Selection**: `loc` (label-based) and `iloc` (integer-based). - **Groupby and Aggregate**: Grouping records by one or more keys and running statistical aggregations (mean, sum, count, etc.).

Deep Dive: Vectorization and Memory Alignment

In numerical computing, **vectorization** replaces explicit loop constructs with array expressions. NumPy enables vectorization by implementing computations in optimized, compiled C code. Under the hood, a NumPy `ndarray` is a contiguous block of memory. Because elements are stored contiguously, modern CPUs can leverage **SIMD** (Single Instruction, Multiple Data) instructions to process multiple operations in a single CPU cycle. In Pandas, data alignment is a fundamental behavior: when operations are performed on two Series or DataFrames, Pandas automatically aligns elements based on index labels. If labels do not overlap, Pandas inserts `NaN` to represent the mismatch, preserving data integrity.

Example Implementation

```
import numpy as np
import pandas as pd

# NumPy Demonstration
arr = np.array([1, 2, 3, 4, 5])
print("Original NumPy array:", arr)
print("Vectorized multiplication (arr * 2):", arr * 2)

# Pandas Demonstration
data = {
    'Product': ['Apple', 'Banana', 'Orange', 'Apple', 'Banana'],
    'Sales': [100, 150, 80, 120, 90]
}
df = pd.DataFrame(data)
print("Pandas DataFrame:")
print(df)

# Groupby example
grouped = df.groupby('Product').sum()
print("Sum of Sales by Product:")
print(grouped)
```

EXECUTION OUTPUT

```
Original NumPy array: [1 2 3 4 5]
Vectorized multiplication (arr * 2): [ 2  4  6  8 10]
Pandas DataFrame:
  Product  Sales
0  Apple    100
1  Banana   150
2  Orange    80
3  Apple    120
4  Banana    90
Sum of Sales by Product:
      Sales
Product
Apple     220
Banana    240
Orange     80
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Create NumPy Array

Create a 1D NumPy array named `arr_easy` containing integers from 1 to 10.

How to solve: Use `np.arange(1, 11)` or `np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])`.

TEMPLATE CODE:

```
import numpy as np
# TODO: Create a 1D NumPy array containing integers from 1 to 10
arr_easy = ____
print(arr_easy)
```

EASY

Exercise 2: Slice NumPy Array

Given the NumPy array `arr_to_slice`, slice the first 5 elements and store the result in `arr_sliced`.

How to solve: Use slice syntax: `arr_to_slice[:5]`.

TEMPLATE CODE:

```
arr_to_slice = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
# TODO: Slice the first 5 elements
arr_sliced = ____
print(arr_sliced)
```

EASY

Exercise 3: Select Pandas Column

Given a Pandas DataFrame `df_easy`, select the column `'Age'` and store it in a variable named `age_column`.

How to solve: Use `df_easy['Age']`.

TEMPLATE CODE:

```
import pandas as pd
df_easy = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]})
# TODO: Select the 'Age' column
age_column = ____
print(age_column)
```

EASY

Exercise 4: Create NumPy Array

Create a 1D NumPy array named `arr_easy` containing integers from 1 to 10.

How to solve: Use Pandas comparison operators (e.g. `df['Age'] > 30`) to generate a boolean mask. Apply this mask to the DataFrame to filter rows, then index using a list of column names: `df[mask][['Name', 'Score']]`.

TEMPLATE CODE:

```
import numpy as np
# TODO: Create a 1D NumPy array containing integers from 1 to 10
arr_easy = ____
print(arr_easy)
```

EASY

Exercise 5: Slice NumPy Array

Given the NumPy array `arr_to_slice`, slice the first 5 elements and store the result in `arr_sliced`.

How to solve: Call `.groupby('Category')` on the DataFrame. Chain the `.agg()` method, passing named arguments to rename columns and specify the aggregation function:

`Avg_Revenue=('Revenue', 'mean'), Total_Quantity=('Quantity', 'sum').`

TEMPLATE CODE:

```
arr_to_slice = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
# TODO: Slice the first 5 elements
arr_sliced = ____
print(arr_sliced)
```

EASY

Exercise 6: Select Pandas Column

Given a Pandas DataFrame `df_easy`, select the column 'Age' and store it in a variable named `age_column`.

How to solve: First, sum elements along rows using `np.sum(A, axis=1, keepdims=True)`. The `keepdims=True` argument is crucial to allow broadcasting division. Divide `$$$` by this sum to get the normalized matrix, then perform matrix multiplication using the `@` operator: `A_norm @ B`.

TEMPLATE CODE:

```
import pandas as pd
df_easy = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]})
# TODO: Select the 'Age' column
age_column = ____
print(age_column)
```

CHAPTER 2

Data Cleaning & Preprocessing

Theory & Foundations

Machine learning algorithms require clean, structured numerical inputs. In raw data, this is rarely the case. In this notebook, we will explore: 1. **Handling Missing Values**: Imputation (filling in missing cells) or removal. 2. **Categorical Encoding**: Converting strings into numerical representations using Ordinal Encoding or One-Hot Encoding. 3. **Feature Scaling**: Centering and scaling distributions using Min-Max scaling or Standardization (Z-score).

1. Missing Values

Common strategies: - Fill with constant (e.g. 0). - Fill with statistics (mean, median, mode). - Predictive imputation (using other features).

2. Encoding Categorical Variables

- **Ordinal Encoding**: Converting categorical labels to integers (0, 1, 2...) when there is an inherent ordering (e.g., Small, Medium, Large).
- **One-Hot Encoding**: Creating binary dummy columns for each category (e.g., Color_Red, Color_Blue) when there is no order.

Deep Dive: Data Integrity and Encoding Trade-offs

Data cleaning is the most critical phase in a machine learning project. When dealing with missing values, **imputation** (filling in gaps) is preferred over removing rows to retain sample size. However, imputing with a simple statistic like the mean can artificially reduce variance. For categorical features, the choice of encoding affects how a model interprets the feature space: - **Ordinal Encoding**: Used when categories have a natural sequence (e.g., small, medium, large). It maps values directly to ordered integers. - **One-Hot Encoding**: Used when categories are nominal (unordered, e.g., colors). It creates dummy columns. Using ordinal encoding on nominal variables is a common mistake; it forces the model to assume a linear relationship where none exists.

Example Implementation

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler

# Example of imputation
s = pd.Series([1, np.nan, 3, np.nan, 5])
print("Imputed with mean:", s.fillna(s.mean()).tolist())

# Example of One-Hot Encoding
df_cat = pd.DataFrame({'Color': ['Red', 'Blue', 'Red']})
onehot = pd.get_dummies(df_cat, columns=['Color'])
print("One-hot Encoded:")
print(onehot)
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Impute Constant Value

Given a Pandas Series `s_easy` with missing values, impute (fill) all NaNs with a constant value of `0.0`. Store the result in `s_filled`.

How to solve: Use `s_easy.fillna(0.0)`.

TEMPLATE CODE:

```
import pandas as pd
import numpy as np
s_easy = pd.Series([1.5, np.nan, 2.5, np.nan, 3.5])
# TODO: Fill NaNs with 0.0
s_filled = ____
print(s_filled)
```

EASY

Exercise 2: Drop Missing Values

Given a Pandas DataFrame `df_miss`, drop all rows containing any missing values and store the result in `df_clean`.

How to solve: Use `df_miss.dropna()`.

TEMPLATE CODE:

```
df_miss = pd.DataFrame({'A': [1, np.nan, 3], 'B': [4, 5, np.nan]})
# TODO: Drop rows with missing values
df_clean = ____
print(df_clean)
```

EASY

Exercise 3: Lowercase Categorical Strings

Given a Pandas Series `s_str`, convert all string values to lowercase. Store the result in `s_lower`.

How to solve: Use `s_str.str.lower()`.

TEMPLATE CODE:

```
s_str = pd.Series(['RED', 'Blue', 'GREEN'])
# TODO: Convert to lowercase
s_lower = ____
print(s_lower)
```

EASY

Exercise 4: Impute Constant Value

Given a Pandas Series `s_easy` with missing values, impute (fill) all NaNs with a constant value of `0.0`. Store the result in `s_filled`.

How to solve: Use the `fillna()` method on the series `s`. Calculate the mean of the series using `s.mean()` and pass it as the argument to fill all missing (NaN) values.

TEMPLATE CODE:

```
import pandas as pd
import numpy as np
s_easy = pd.Series([1.5, np.nan, 2.5, np.nan, 3.5])
# TODO: Fill NaNs with 0.0
s_filled = ____
print(s_filled)
```

EASY

Exercise 5: Drop Missing Values

Given a Pandas DataFrame `df_miss`, drop all rows containing any missing values and store the result in `df_clean`.

How to solve: Use the `map()` method with a dictionary mapping categories to numbers for the ordinal column: `df['Size'].map({'Small': 0, 'Medium': 1, 'Large': 2})`. For nominal encoding, use `pd.get_dummies(df, columns=['Color'])`.

TEMPLATE CODE:

```
df_miss = pd.DataFrame({'A': [1, np.nan, 3], 'B': [4, 5, np.nan]})
# TODO: Drop rows with missing values
df_clean = ____
print(df_clean)
```

EASY

Exercise 6: Lowercase Categorical Strings

Given a Pandas Series `s_str`, convert all string values to lowercase. Store the result in `s_lower`.

How to solve: Find the 10th and 90th percentiles of the array using `np.percentile(arr, 10)` and `np.percentile(arr, 90)`. Use `np.clip(arr, p10, p90)` to clip values, then apply Min-Max scaling: `(clipped - clipped.min()) / (clipped.max() - clipped.min()) * (max_val - min_val) + min_val`.

TEMPLATE CODE:

```
s_str = pd.Series(['RED', 'Blue', 'GREEN'])
# TODO: Convert to lowercase
s_lower = ____
print(s_lower)
```

CHAPTER 3

Data Visualization & Plotting

Theory & Foundations

Data visualization is key to exploratory data analysis (EDA). It allows you to:

- Identify patterns and distributions.
- Discover relationships between variables.
- Detect outliers and anomalies.

In Python, the primary plotting libraries are:

1. **Matplotlib**: A low-level plotting library providing full control over the figure, axes, labels, and ticks.
2. **Seaborn**: A high-level data visualization library built on top of Matplotlib. It provides visually appealing default styles and handles integration with Pandas DataFrames natively.

Deep Dive: Visual Analytics and Correlation

Data visualization is an exploratory tool that reveals structural patterns, anomalies, and relationships. Histograms plot the frequency distribution of continuous variables, showing skewness, modality (unimodal, bimodal), and potential outliers. Scatter plots visualize bivariate relationships, allowing us to eyeball correlations and linearities. The Pearson correlation coefficient r measures the linear relationship between variables, ranging from -1 to +1. A correlation matrix heat map helps identify **multicollinearity** (high correlation between feature variables), which can destabilize linear models and make coefficient interpretations unreliable.

Example Implementation

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# Set plot style
sns.set_theme(style="darkgrid")

# Create sample data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Generate demonstration plot
plt.figure(figsize=(8, 4))
plt.plot(x, y, label="Sin(x)", color="purple", lw=2)
plt.title("Matplotlib Demo Line Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Matplotlib Line Plot

Create a simple line plot using `plt.plot()` with `x` on the horizontal axis and `y` on the vertical axis.

How to solve: Call `plt.plot(x, y)`.

TEMPLATE CODE:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
# TODO: Plot x vs y

plt.show()
```

EASY

Exercise 2: Add Title to Plot

Create a line plot of `x` vs `y` and set the title of the plot to `'My First Plot'`. Add X-axis label `'X Axis'` and Y-axis label `'Y Axis'`.

How to solve: Use `plt.title('My First Plot')`, `plt.xlabel('X Axis')`, and `plt.ylabel('Y Axis')`.

TEMPLATE CODE:

```
x = [1, 2, 3, 4]
y = [1, 4, 9, 16]
plt.plot(x, y)
# TODO: Add title and axis labels

plt.show()
```

EASY

Exercise 3: Seaborn Scatter Plot

Create a scatter plot using Seaborn's `sns.scatterplot()` with the given coordinates.

How to solve: Use `sns.scatterplot(x=x_coords, y=y_coords)`.

TEMPLATE CODE:

```
import seaborn as sns
x_coords = [1, 2, 3, 4]
y_coords = [10, 15, 13, 18]
# TODO: Draw a scatter plot
_____
plt.show()
```

EASY

Exercise 4: Matplotlib Line Plot

Create a simple line plot using `plt.plot()` with `x` on the horizontal axis and `y` on the vertical axis.

How to solve: Use Matplotlib's `plt.hist(data, bins=20)`. Add labels and titles using `plt.xlabel('Value')`, `plt.ylabel('Frequency')`, and `plt.title('Feature Distribution')`. Call `plt.show()` to display.

TEMPLATE CODE:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
# TODO: Plot x vs y
_____
plt.show()
```

EASY

Exercise 5: Add Title to Plot

Create a line plot of x vs y and set the title of the plot to 'My First Plot'. Add X-axis label 'X Axis' and Y-axis label 'Y Axis'.

How to solve: Use Seaborn's `sns.lmplot(x='x', y='y', hue='group', data=df)`. The `lmplot` function automatically fits and plots a linear regression line for each group defined by the `hue` column.

TEMPLATE CODE:

```
x = [1, 2, 3, 4]
y = [1, 4, 9, 16]
plt.plot(x, y)
# TODO: Add title and axis labels
____
____
____
plt.show()
```

EASY

Exercise 6: Seaborn Scatter Plot

Create a scatter plot using Seaborn's `sns.scatterplot()` with the given coordinates.

How to solve: Compute the correlation matrix with `corr = df.corr()`. Filter weak correlations by setting absolute values less than 0.3 to NaN: `corr[corr.abs() < 0.3] = np.nan`. Plot using `sns.heatmap(corr, annot=True, cmap='coolwarm')`.

TEMPLATE CODE:

```
import seaborn as sns
x_coords = [1, 2, 3, 4]
y_coords = [10, 15, 13, 18]
# TODO: Draw a scatter plot
____
plt.show()
```


CHAPTER 4

Regression (Linear & Logistic)

Theory & Foundations

Regression is a supervised learning task where the model predicts continuous target outputs. In this notebook, we cover: 1. **Simple and Multiple Linear Regression**: Minimizing Mean Squared Error (MSE). 2. **Gradient Descent Optimization**: How parameters are learned iteratively. 3. **Logistic Regression**: Used for classification, predicting probabilities using the Sigmoid activation function:

$$\sigma(z) = (1)/(1 + e^{-z})$$

1. **Regularization**: L_1 regularization (Lasso, which drives some weights to exactly 0) and L_2 regularization (Ridge, which shrinks weights).

Deep Dive: Loss Optimization and Sparse Regularization

Linear Regression fits a hyperplane that minimizes the Mean Squared Error (MSE):

$$L = (1)/(N) \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i - b)^2$$

Logistic Regression extends this to binary classification by passing the linear output through the Sigmoid activation:

$$P(y=1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = (1)/(1 + e^{-(\mathbf{w}^T \mathbf{x} + b)})$$

To fit these models, **Gradient Descent** iteratively calculates partial derivatives of the loss function with respect to weights, updating them in the direction of steepest descent. When models overfit, **regularization** penalizes large weights: - **L1 (Lasso)**: Adds an absolute weight penalty ($\|\mathbf{w}\|_1$). This results in **sparsity**, setting less important feature weights to exactly zero, thus acting as feature selection. - **L2 (Ridge)**: Adds a squared weight penalty ($\|\mathbf{w}\|_2^2$). It shrinks weights close to zero but keeps all features.

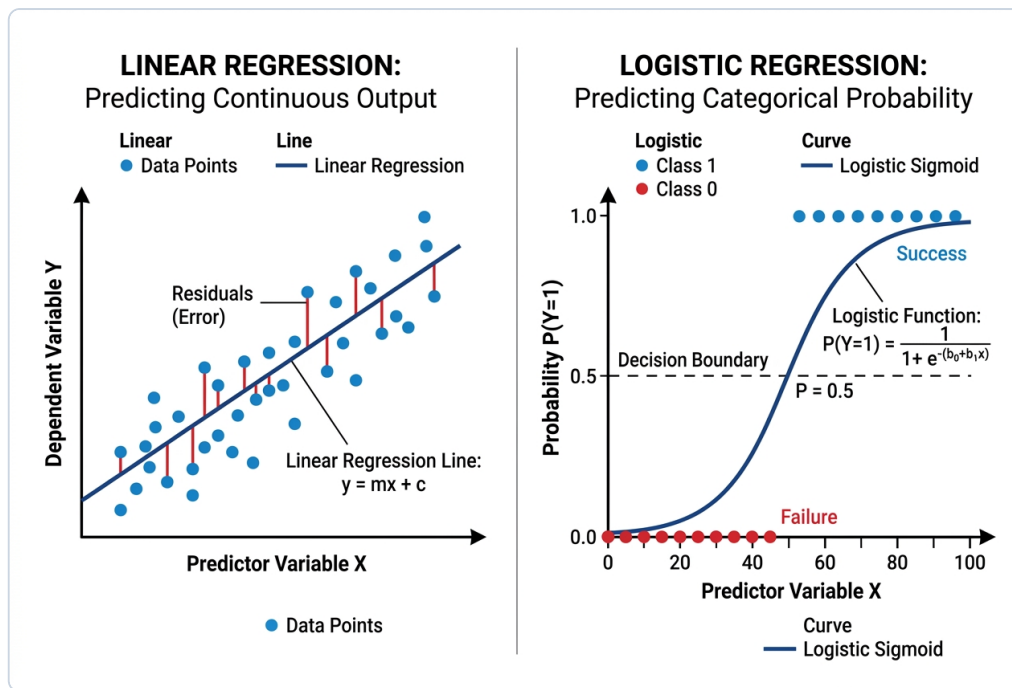


Figure 4.1: Linear Regression fitting vs. Logistic Regression Sigmoid curve.

Example Implementation

```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error

# Quick scikit-learn regression example
X = np.array([[1], [2], [3], [4]])
y = np.array([2.1, 3.9, 6.1, 8.0])

reg = LinearRegression().fit(X, y)
print("Coefficients:", reg.coef_)
print("Intercept:", reg.intercept_)
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Import Linear Regression

Import `LinearRegression` from `sklearn.linear_model` and instantiate it. Store the instance in a variable named `lr_model`.

How to solve: Use: `from sklearn.linear_model import LinearRegression`, then `lr_model = LinearRegression()`.

TEMPLATE CODE:

```
# TODO: Import LinearRegression and instantiate it
lr_model = _____
print(lr_model)
```

EASY

Exercise 2: Compute Prediction Error

Given the true values `y_true` and predicted values `y_pred`, calculate the mean squared error (MSE) using NumPy. Store it in `mse`.

How to solve: Formula: `np.mean((y_true - y_pred) ** 2)`.

TEMPLATE CODE:

```
import numpy as np
y_true = np.array([2.0, 3.0, 4.0])
y_pred = np.array([1.9, 3.2, 3.8])
# TODO: Compute Mean Squared Error using NumPy
mse = _____
print(mse)
```

EASY

Exercise 3: Train Linear Regression

Fit the given `LinearRegression` model `model` using features `x` and labels `y`.

How to solve: Call `model.fit(X, y)`.

TEMPLATE CODE:

```
from sklearn.linear_model import LinearRegression
X = np.array([[1], [2], [3]])
y = np.array([2, 4, 6])
model = LinearRegression()
# TODO: Train the model

_____
print("Intercept:", model.intercept_)
```

EASY

Exercise 4: Import Linear Regression

Import `LinearRegression` from `sklearn.linear_model` and instantiate it. Store the instance in a variable named `lr_model`.

How to solve: Import `LinearRegression` from `sklearn.linear_model`. Instantiate it, fit the model on training data using `.fit(X_train, y_train)`, and make predictions on the test set using `.predict(X_test)`.

TEMPLATE CODE:

```
# TODO: Import LinearRegression and instantiate it
lr_model = _____
print(lr_model)
```

EASY

Exercise 5: Compute Prediction Error

Given the true values `y_true` and predicted values `y_pred`, calculate the mean squared error (MSE) using NumPy. Store it in `mse`.

How to solve: Initialize `w` and `b` to 0.0. In a loop over epochs, calculate predictions: `pred = w * X + b`. Calculate gradients: `dw = -2/N * np.sum(X * (y - pred))` and `db = -2/N * np.sum(y - pred)`. Update weights: `w -= lr * dw` and `b -= lr * db`.

TEMPLATE CODE:

```
import numpy as np
y_true = np.array([2.0, 3.0, 4.0])
y_pred = np.array([1.9, 3.2, 3.8])
# TODO: Compute Mean Squared Error using NumPy
mse = _____
print(mse)
```

EASY

Exercise 6: Train Linear Regression

Fit the given `LinearRegression` model `model` using features `x` and labels `y`.

How to solve: Train `LogisticRegression(penalty='l1', solver='liblinear', C=0.1)` for L1 and `LogisticRegression(penalty='l2', C=0.1)` for L2. Extract coefficients using `model.coef_[0]`.

TEMPLATE CODE:

```
from sklearn.linear_model import LinearRegression
X = np.array([[1], [2], [3]])
y = np.array([2, 4, 6])
model = LinearRegression()
# TODO: Train the model

print("Intercept:", model.intercept_)
```

CHAPTER 5

Decision Trees & Random Forests

Theory & Foundations

Decision Trees and Random Forests are widely used non-parametric models for classification and regression.

1. Decision Trees

A Decision Tree splits the data recursively into sub-nodes. - **Split Criterion**: Calculated using Gini Impurity or Entropy (for classification), or Variance Reduction (for regression). - **Overfitting**: Trees can easily grow to fit noise in the training set. We control this using constraints like `max_depth`, `min_samples_split`, or `min_samples_leaf`.

2. Random Forests

An ensemble model that builds a forest of randomized decision trees. - **Bagging (Bootstrap Aggregating)**: Each tree is trained on a random bootstrap sample of the training data. - **Feature Randomization**: At each split point, only a random subset of features is considered. - **Averaging**: The final prediction is a vote (classification) or average (regression) of all tree outputs.

Deep Dive: Ensemble Synergy & Bias-Variance Trade-off

Decision Trees partition the feature space recursively by choosing splits that maximize information gain or minimize **Gini Impurity**:

$$\text{Gini} = 1 - \sum_{k=1}^C p_k^2$$

Deep trees partition the space too specifically, leading to low bias but high variance (overfitting). Constraining `max_depth` limits the splits to create more general rules. **Random Forests** solve the high variance of individual trees by building an ensemble of diverse trees. Using **Bootstrap Aggregating (Bagging)**, each tree is trained on a random sample of the data (with replacement), and splits are evaluated on a random subset of features. The trees vote (classification) or average (regression) to yield a model with low variance.

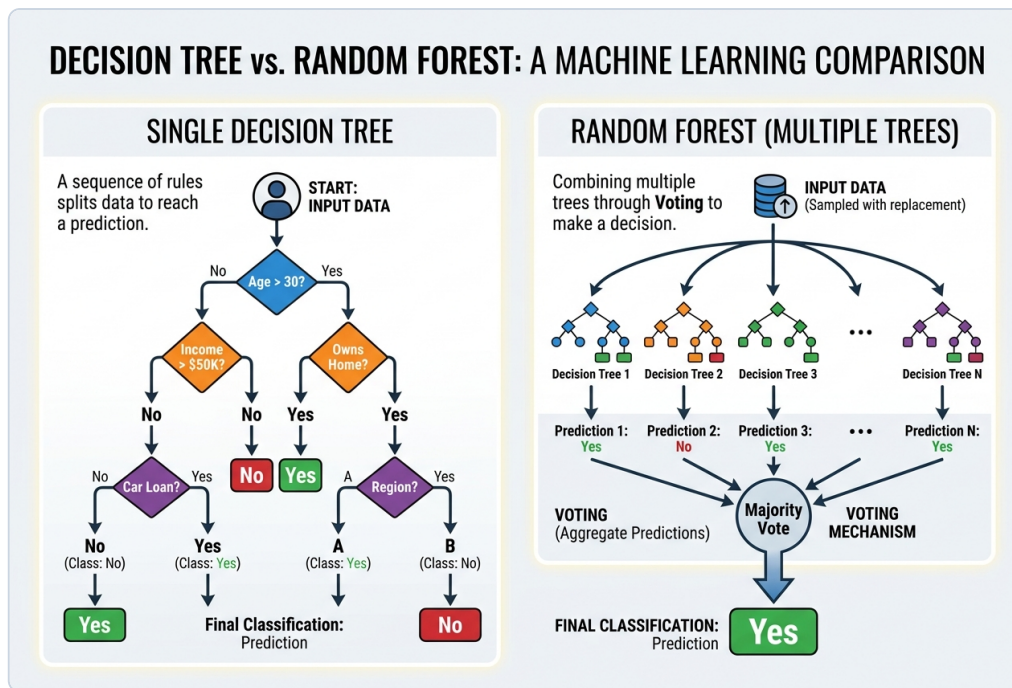


Figure 5.1: Structure of a Single Decision Tree vs. Random Forest Voting.

Example Implementation

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Demonstration
iris = load_iris()
X_tr, X_te, y_tr, y_te = train_test_split(iris.data, iris.target, test_size=0.2, r
andom_state=42)
dt = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X_tr, y_tr)
print("Decision Tree Train Acc:", dt.score(X_tr, y_tr))
print("Decision Tree Test Acc:", dt.score(X_te, y_te))
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Import Decision Tree

Import `DecisionTreeClassifier` from `sklearn.tree` and instantiate it setting `max_depth=3`. Store the instance in a variable named `dt_easy`.

How to solve: Use: `from sklearn.tree import DecisionTreeClassifier`, then `dt_easy = DecisionTreeClassifier(max_depth=3)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate DecisionTreeClassifier with max_depth=3
dt_easy = ____
print(dt_easy)
```

EASY

Exercise 2: Predict Labels

Given a trained model `model` and new samples `X_new`, predict class labels and store the predictions in `predictions`.

How to solve: Use `model.predict(X_new)`.

TEMPLATE CODE:

```
from sklearn.tree import DecisionTreeClassifier
X = np.array([[1, 2], [3, 4], [5, 6]])
y = np.array([0, 0, 1])
model = DecisionTreeClassifier(random_state=42).fit(X, y)
X_new = np.array([[2, 3], [4, 5]])
# TODO: Predict labels for X_new
predictions = ____
print(predictions)
```


EASY

Exercise 3: Calculate Accuracy Score

Calculate the classification accuracy of predicted labels `y_pred` against true labels `y_true`. Store it in `acc`.

How to solve: Call `accuracy_score(y_true, y_pred)`.

TEMPLATE CODE:

```
from sklearn.metrics import accuracy_score
y_true = np.array([0, 1, 1, 0])
y_pred = np.array([0, 1, 0, 0])
# TODO: Calculate accuracy score
acc = ____
print(acc)
```

EASY

Exercise 4: Import Decision Tree

Import `DecisionTreeClassifier` from `sklearn.tree` and instantiate it setting `max_depth=3`. Store the instance in a variable named `dt_easy`.

How to solve: Import `DecisionTreeClassifier` from `sklearn.tree`. Instantiate it setting `max_depth=3` and `random_state=42`, then call `.fit(X_train, y_train)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate DecisionTreeClassifier with max_depth=3
dt_easy = ____
print(dt_easy)
```

EASY

Exercise 5: Predict Labels

Given a trained model `model` and new samples `X_new`, predict class labels and store the predictions in `predictions`.

How to solve: Train `RandomForestClassifier(n_estimators=100, max_depth=5, min_samples_split=4, random_state=42)`. Call `.predict(X_test)` and calculate the accuracy score using `accuracy_score(y_test, predictions)`.

TEMPLATE CODE:

```
from sklearn.tree import DecisionTreeClassifier
X = np.array([[1, 2], [3, 4], [5, 6]])
y = np.array([0, 0, 1])
model = DecisionTreeClassifier(random_state=42).fit(X, y)
X_new = np.array([[2, 3], [4, 5]])
# TODO: Predict labels for X_new
predictions = ____
print(predictions)
```

EASY

Exercise 6: Calculate Accuracy Score

Calculate the classification accuracy of predicted labels `y_pred` against true labels `y_true`. Store it in `acc`.

How to solve: Get feature importances using `rf_model.feature_importances_`. Find indices where importance is greater than 0.1 using `np.where(importances > 0.1)[0]`, and filter training columns: `X_train[:, selected_indices]`.

TEMPLATE CODE:

```
from sklearn.metrics import accuracy_score
y_true = np.array([0, 1, 1, 0])
y_pred = np.array([0, 1, 0, 0])
# TODO: Calculate accuracy score
acc = ____
print(acc)
```

CHAPTER 6

Support Vector Machines & Naive Bayes

Theory & Foundations

In this notebook, we cover two classical and powerful classification algorithms:

1. Support Vector Machines (SVM)

SVM aims to find the optimal hyperplane that separates classes with the maximum margin. - **Support Vectors**: The data points closest to the decision boundary. - **Kernel Trick**: Projects data into a higher-dimensional space to make it linearly separable. Common kernels: Linear, Polynomial, and Radial Basis Function (RBF). - **Hyperparameters**: - c : Controls the trade-off between maximizing margin and minimizing classification errors.

2. Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' Theorem:

$$P(y | x_1 \dots x_n) = (P(y) P(x_1 \dots x_n | y)) / (P(x_1 \dots x_n))$$

It makes the **"naive" assumption** that features are conditionally independent given the class label. Very popular for text classification and spam detection.

Deep Dive: Maximum Margin Separation & Naive Bayes Probabilities

Support Vector Machines (SVM) find the optimal hyperplane that separates classes while maximizing the **margin**—the distance between the boundary and the closest data points (the support vectors). For non-linear datasets, SVMs project data into a higher-dimensional space where classes are linearly separable. This is done using a **Kernel function** (e.g., Radial Basis Function, RBF) without explicitly computing high-dimensional coordinates, saving computational resources. **Naive Bayes** is a probabilistic classifier based on Bayes' Theorem:

$$P(y|x) = (P(x|y) P(y)) / (P(x))$$

It is called "naive" because it assumes all features are conditionally independent given the class label. Despite this simplification, it is highly effective and fast for high-dimensional text datasets.

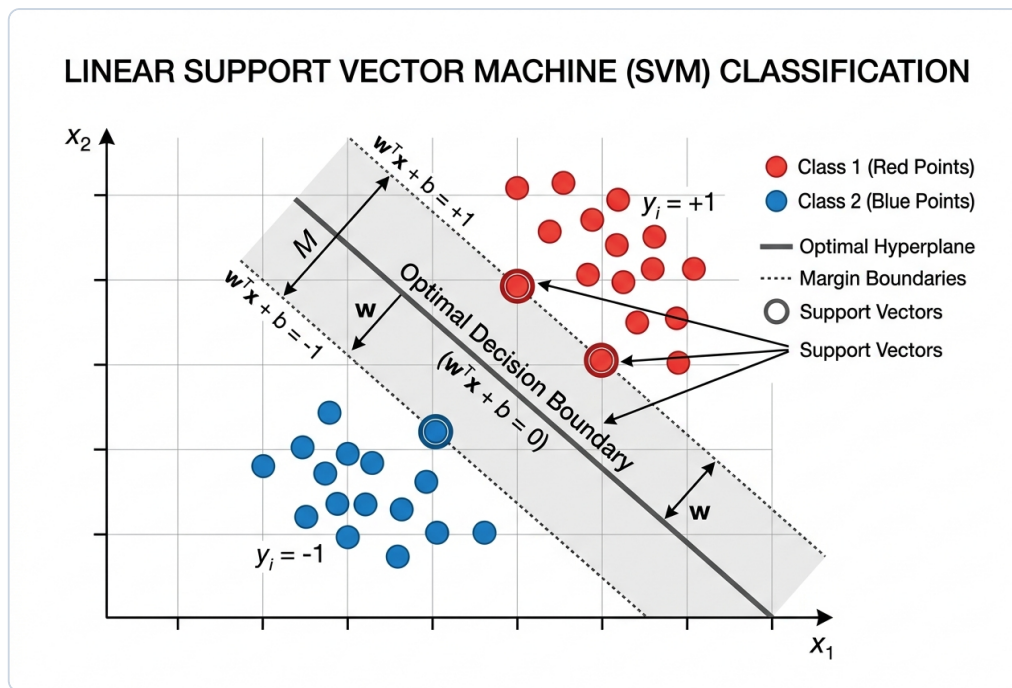


Figure 6.1: SVM classification, showing margin boundaries and support vectors.

Example Implementation

```
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
import numpy as np

# Sample SVM classification
X = np.array([[1, 2], [2, 3], [3, 1], [6, 5], [7, 8], [8, 6]])
y = np.array([0, 0, 0, 1, 1, 1])

clf = SVC(kernel='linear').fit(X, y)
print("Support Vectors: ", clf.support_vectors_)
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Import Naive Bayes

Import `MultinomialNB` from `sklearn.naive_bayes` and instantiate it. Store it in a variable named `nb_easy`.

How to solve: Use: `from sklearn.naive_bayes import MultinomialNB`, then `nb_easy = MultinomialNB()`.

TEMPLATE CODE:

```
# TODO: Import and instantiate MultinomialNB
nb_easy = ____
print(nb_easy)
```

EASY

Exercise 2: Linear SVM Instantiation

Import `SVC` from `sklearn.svm` and instantiate it with a `'linear'` kernel. Store the model in `svm_easy`.

How to solve: Use: `from sklearn.svm import SVC`, then `svm_easy = SVC(kernel='linear')`.

TEMPLATE CODE:

```
# TODO: Import SVC and instantiate with kernel='linear'
svm_easy = ____
print(svm_easy)
```

EASY

Exercise 3: Compute Probability Output

For a trained Support Vector Machine model `model`, predict probabilities on `X_test`. Store the result in `probs`.

How to solve: Use `model.predict_proba(X_test)`.

TEMPLATE CODE:

```
from sklearn.svm import SVC
# Assuming probability support is enabled
X = np.array([[1], [2], [3], [4]])
y = np.array([0, 0, 1, 1])
model = SVC(kernel='linear', probability=True).fit(X, y)
X_test = np.array([[2.5]])
# TODO: Predict probability
probs = ____
print(probs)
```

EASY

Exercise 4: Import Naive Bayes

Import `MultinomialNB` from `sklearn.naive_bayes` and instantiate it. Store it in a variable named `nb_easy`.

How to solve: Import `MultinomialNB` from `sklearn.naive_bayes`. Instantiate it with default parameters, and fit it on the TF-IDF representation of the text: `nb_model.fit(X_train, y_train)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate MultinomialNB
nb_easy = ____
print(nb_easy)
```

EASY

Exercise 5: Linear SVM Instantiation

Import `SVC` from `sklearn.svm` and instantiate it with a `'linear'` kernel. Store the model in `svm_easy`.

How to solve: Train `SVC(kernel='linear')` on the dataset. The indices of the support vectors are stored in the `support_` attribute of the fitted model.

TEMPLATE CODE:

```
# TODO: Import SVC and instantiate with kernel='linear'
svm_easy = _____
print(svm_easy)
```

EASY

Exercise 6: Compute Probability Output

For a trained Support Vector Machine model `model`, predict probabilities on `x_test`. Store the result in `probs`.

How to solve: Train an SVM with a linear kernel: `SVC(kernel='linear')`, and another with an RBF kernel: `SVC(kernel='rbf')`. Compare their accuracy scores on the non-linear test dataset.

TEMPLATE CODE:

```
from sklearn.svm import SVC
# Assuming probability support is enabled
X = np.array([[1], [2], [3], [4]])
y = np.array([0, 0, 1, 1])
model = SVC(kernel='linear', probability=True).fit(X, y)
X_test = np.array([[2.5]])
# TODO: Predict probability
probs = _____
print(probs)
```

CHAPTER 7

Unsupervised Learning & Clustering

Theory & Foundations

Unsupervised learning operates on unlabeled data to find hidden groupings or reduce dimensions. In this notebook, we cover: 1. **Principal Component Analysis (PCA)**: Linear dimensionality reduction. It projects data onto orthogonal axes that maximize variance. 2. **K-Means Clustering**: Partitions data into k distinct clusters by minimizing the sum of squared distances (inertia) between points and centroids. 3. **Elbow Method**: A technique to identify the optimal number of clusters k by plotting inertia vs. k .

Deep Dive: Dimensional Projection & Centroid Convergence

Principal Component Analysis (PCA) is an unsupervised technique that finds orthogonal axes (Principal Components) along which the variance of the data is maximized. It projects high-dimensional data into a lower-dimensional space while retaining as much informational variance as possible. **K-Means** is a clustering algorithm that partitions data into k groups. It iteratively: 1. Assigns each point to the nearest centroid. 2. Updates centroids to the mean of assigned points. The algorithm converges when assignments stop changing. The **Elbow Method** evaluates clustering quality by plotting the sum of squared distances (inertia) against different values of k . The "elbow" point represents the optimal trade-off between clustering detail and simplicity.

Example Implementation

```
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load data
iris = load_iris()
X = iris.data

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print("Original shape:", X.shape)
print("PCA shape:", X_pca.shape)
```


Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Import PCA

Import `PCA` from `sklearn.decomposition` and instantiate it setting `n_components=2`. Store the instance in `pca_easy`.

How to solve: Use: `from sklearn.decomposition import PCA`, then `pca_easy = PCA(n_components=2)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate PCA with 2 components
pca_easy = ____
print(pca_easy)
```

EASY

Exercise 2: KMeans Instantiation

Import `KMeans` from `sklearn.cluster` and instantiate it with `n_clusters=3`. Store the model in `kmeans_easy`.

How to solve: Use: `from sklearn.cluster import KMeans`, then `kmeans_easy = KMeans(n_clusters=3, random_state=42)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate KMeans with 3 clusters
kmeans_easy = ____
print(kmeans_easy)
```

EASY

Exercise 3: Predict Cluster Labels

Given a fitted KMeans model `model` and features `X`, predict the cluster label assignments. Store the predictions in `labels`.

How to solve: Use `model.predict(X)` or `model.labels_`.

TEMPLATE CODE:

```
from sklearn.cluster import KMeans
X = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])
model = KMeans(n_clusters=2, random_state=42).fit(X)
# TODO: Predict cluster labels
labels = ____
print(labels)
```

EASY

Exercise 4: Import PCA

Import PCA from `sklearn.decomposition` and instantiate it setting `n_components=2`. Store the instance in `pca_easy`.

How to solve: Import PCA from `sklearn.decomposition`. Fit it and transform the data using `pca.fit_transform(X)`, setting `n_components=2`.

TEMPLATE CODE:

```
# TODO: Import and instantiate PCA with 2 components
pca_easy = ____
print(pca_easy)
```

EASY

Exercise 5: KMeans Instantiation

Import `KMeans` from `sklearn.cluster` and instantiate it with `n_clusters=3`. Store the model in `kmeans_easy`.

How to solve: Loop over cluster counts k from 1 to 10. Fit `KMeans` for each k , extract the inertia using `kmeans.inertia_`, and plot these values to find the elbow point.

TEMPLATE CODE:

```
# TODO: Import and instantiate KMeans with 3 clusters
kmeans_easy = ____
print(kmeans_easy)
```

EASY

Exercise 6: Predict Cluster Labels

Given a fitted KMeans model `model` and features `x`, predict the cluster label assignments. Store the predictions in `labels`.

How to solve: In the loop: 1. Calculate distances from each point to each centroid. 2. Assign points to the closest centroid using `np.argmin(distances, axis=1)`. 3. Update centroids by computing the mean of points in each cluster: `np.mean(X[assignments == i], axis=0)`.

TEMPLATE CODE:

```
from sklearn.cluster import KMeans
X = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])
model = KMeans(n_clusters=2, random_state=42).fit(X)
# TODO: Predict cluster labels
labels = ____
print(labels)
```

CHAPTER 8

Feature Engineering & Selection

Theory & Foundations

Feature engineering is the process of using domain knowledge to create new features that help machine learning algorithms perform better. Feature selection helps discard redundant or noisy features. In this notebook, we cover: 1. **Polynomial Features**: Capture non-linear interactions. 2. **Cyclical Features**: Handle cyclic temporal data (hours, months) using sine/cosine transforms. 3. **Recursive Feature Elimination (RFE)**: Fit model iteratively, removing the least important features.

Deep Dive: Feature Space Expansion and Selection

Machine learning models perform best when features match the underlying patterns. **Polynomial features** expand the feature space by creating multiplicative interactions (e.g., x_1^2 , $x_1 x_2$), allowing linear models to fit non-linear curves. **Cyclical feature encoding** maps periodic variables (e.g., hour, month) using trigonometric functions:

$$x_{\sin} = \sin((2 \pi x)/(\text{period})), x_{\cos} = \cos((2 \pi x)/(\text{period}))$$

This preserves the temporal proximity of end and start boundaries (e.g., 23:00 and 00:00).

Recursive Feature Elimination (RFE) is a wrapper-based feature selection method. It trains a model, ranks features by importance or coefficients, prunes the least important feature, and repeats until the target number of features is reached.

Example Implementation

```
from sklearn.preprocessing import PolynomialFeatures
import pandas as pd
import numpy as np

# Example: Polynomial features
X = np.array([[2, 3]])
poly = PolynomialFeatures(degree=2, include_bias=False)
print("Poly Features of [2, 3]:", poly.fit_transform(X))
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Add Interaction Column

Given a Pandas DataFrame `df`, create a new column named `'A_B'` by multiplying column `'A'` and column `'B'`.

How to solve: Set `df['A_B'] = df['A'] * df['B']`.

TEMPLATE CODE:

```
import pandas as pd
df = pd.DataFrame({'A': [2, 3, 4], 'B': [5, 10, 15]})
# TODO: Create column 'A_B'

print(df)
```

EASY

Exercise 2: Apply Log Transform

Apply a logarithmic transformation to the `'Skewed'` column of `df`. Store the log-transformed values in a new column named `'Log_Skewed'`.

How to solve: Use `np.log(df['Skewed'])`.

TEMPLATE CODE:

```
import numpy as np
df = pd.DataFrame({'Skewed': [10.0, 100.0, 1000.0]})
# TODO: Create column 'Log_Skewed'

print(df)
```

EASY

Exercise 3: Polynomial Squared Feature

Create a new feature 'x_squared' in df by raising column 'x' to the second power.

How to solve: Set `df['x_squared'] = df['x'] ** 2`.

TEMPLATE CODE:

```
df = pd.DataFrame({'x': [1, 2, 3, 4]})
# TODO: Create 'x_squared' column
_____
print(df)
```

EASY

Exercise 4: Add Interaction Column

Given a Pandas DataFrame df, create a new column named 'A_B' by multiplying column 'A' and column 'B'.

How to solve: Import `PolynomialFeatures` from `sklearn.preprocessing`. Fit and transform features with `degree=2` and `include_bias=False` to generate interaction and polynomial columns.

TEMPLATE CODE:

```
import pandas as pd
df = pd.DataFrame({'A': [2, 3, 4], 'B': [5, 10, 15]})
# TODO: Create column 'A_B'
_____
print(df)
```

EASY

Exercise 5: Apply Log Transform

Apply a logarithmic transformation to the 'Skewed' column of `df`. Store the log-transformed values in a new column named 'Log_Skewed'.

How to solve: Map the cyclical hour values to sine and cosine features: `sin_hour = np.sin(2 * np.pi * df['Hour'] / 24)` and `cos_hour = np.cos(2 * np.pi * df['Hour'] / 24)`.

TEMPLATE CODE:

```
import numpy as np
df = pd.DataFrame({'Skewed': [10.0, 100.0, 1000.0]})
# TODO: Create column 'Log_Skewed'

_____
print(df)
```

EASY

Exercise 6: Polynomial Squared Feature

Create a new feature 'x_squared' in `df` by raising column 'x' to the second power.

How to solve: Import `RFE` from `sklearn.feature_selection`. Pass a base estimator (like Logistic Regression) and specify the number of features to select, then call `.fit(X, y)`.

TEMPLATE CODE:

```
df = pd.DataFrame({'x': [1, 2, 3, 4]})
# TODO: Create 'x_squared' column

_____
print(df)
```

CHAPTER 9

Model Evaluation & Hyperparameter Tuning

Theory & Foundations

Evaluating and tuning models is crucial to verify they generalize to unseen data. In this notebook, we cover: 1. **Cross-Validation**: K-Fold cross validation splits the dataset into K parts, using K-1 for training and 1 for validation in rotation. 2. **Hyperparameter Tuning**: Searching the parameter space using Grid Search (brute force) or Random Search (stochastic). 3. **Classification Threshold Tuning**: Finding the optimal classification probability cutoff to maximize metrics like the F1-score.

Deep Dive: Model Validation and Evaluation Diagnostics

Evaluating models on training data leads to optimistic bias. **K-Fold Cross-Validation** splits data into k non-overlapping folds. The model is trained on k-1 folds and validated on the remaining fold, repeating k times. This provides a robust distribution of performance. For hyperparameter tuning, **Grid Search** searches every combination in a specified grid, guaranteeing the best parameters are found at high computational cost. **Random Search** samples hyperparameter combinations randomly, covering a wider range in less time. In classification, standard accuracy fails on imbalanced data. We evaluate **F1-Score** (harmonic mean of precision and recall) and adjust the classification probability threshold to optimize the model for specific business requirements.

Example Implementation

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, random_state=42)
rf = RandomForestClassifier(n_estimators=10, random_state=42)
scores = cross_val_score(rf, X, y, cv=5)
print("CV Scores:", scores)
print("Mean CV Score:", scores.mean())
```


Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Train-Test Split

Split features x and labels y into training and testing sets (80% train, 20% test). Store outputs in x_{train} , x_{test} , y_{train} , y_{test} . Set `random_state=42`.

How to solve: Use `train_test_split(X, y, test_size=0.2, random_state=42)`.

TEMPLATE CODE:

```
from sklearn.model_selection import train_test_split
X = np.arange(10).reshape((5, 2))
y = np.array([0, 1, 0, 1, 0])
# TODO: Train-test split (test_size=0.2, random_state=42)
X_train, X_test, y_train, y_test = ____
print(X_train.shape)
```

EASY

Exercise 2: KFold Instantiation

Import `KFold` from `sklearn.model_selection` and instantiate it with `n_splits=5`, `shuffle=True`, and `random_state=42`. Store it in `kf`.

How to solve: Use: `from sklearn.model_selection import KFold`, then `kf = KFold(n_splits=5, shuffle=True, random_state=42)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate KFold with 5 splits
kf = ____
print(kf)
```

EASY

Exercise 3: Calculate Precision Score

Calculate the classification precision score of `y_pred` against `y_true`. Store it in `precision`.

How to solve: Call `precision_score(y_true, y_pred)`.

TEMPLATE CODE:

```
from sklearn.metrics import precision_score
y_true = np.array([0, 1, 1, 0])
y_pred = np.array([0, 1, 0, 0])
# TODO: Calculate precision score
precision = ____
print(precision)
```

EASY

Exercise 4: Train-Test Split

Split features `x` and labels `y` into training and testing sets (80% train, 20% test). Store outputs in `X_train`, `X_test`, `y_train`, `y_test`. Set `random_state=42`.

How to solve: Import `cross_val_score` from `sklearn.model_selection`. Pass the classifier, features, labels, and set the number of folds (e.g. `cv=5`).

TEMPLATE CODE:

```
from sklearn.model_selection import train_test_split
X = np.arange(10).reshape((5, 2))
y = np.array([0, 1, 0, 1, 0])
# TODO: Train-test split (test_size=0.2, random_state=42)
X_train, X_test, y_train, y_test = ____
print(X_train.shape)
```

EASY

Exercise 5: KFold Instantiation

Import `KFold` from `sklearn.model_selection` and instantiate it with `n_splits=5`, `shuffle=True`, and `random_state=42`. Store it in `kf`.

How to solve: Use `GridSearchCV` to search all parameter combinations, and `RandomizedSearchCV` to search a random subset of parameters. Measure and compare execution time and accuracy.

TEMPLATE CODE:

```
# TODO: Import and instantiate KFold with 5 splits
kf = _____
print(kf)
```

EASY

Exercise 6: Calculate Precision Score

Calculate the classification precision score of `y_pred` against `y_true`. Store it in `precision`.

How to solve: Use `model.predict_proba(X_test)[:, 1]` to get probabilities. Loop through thresholds, convert probabilities to binary predictions, and calculate the F1-score to find the best threshold.

TEMPLATE CODE:

```
from sklearn.metrics import precision_score
y_true = np.array([0, 1, 1, 0])
y_pred = np.array([0, 1, 0, 0])
# TODO: Calculate precision score
precision = _____
print(precision)
```

CHAPTER 10

Introduction to Deep Learning

Theory & Foundations

Deep Learning builds models using Artificial Neural Networks (ANN). In this notebook, we cover: 1.

Perceptron: The simplest neural network block, modeling a single neuron:

$$y = \text{activation}(\mathbf{w} \cdot \mathbf{x} + b)$$

1. **Multi-Layer Perceptron (MLP):** A feedforward network with one or more hidden layers.
2. **Activation Functions:** Introduce non-linearities. Common ones are ReLU, Sigmoid, and Tanh.
3. **Backpropagation:** Adjusts weights in the network using gradients computed via the chain rule.

Deep Dive: Deep Neural Architecture & Backpropagation

A **Perceptron** is the basic unit of a neural network. It computes a weighted sum of inputs, adds a bias, and passes the output through a step function. A single perceptron can only learn linear boundaries. To learn non-linear patterns, **Multi-Layer Perceptrons (MLPs)** stack layers of neurons. Hidden layers use non-linear activation functions (like Sigmoid or ReLU) to project data into non-linear spaces. Neural networks learn via **Backpropagation**: 1. **Feedforward:** Input passes through layers to generate predictions. 2. **Loss Calculation:** Computes error compared to targets. 3. **Backpropagation:** The error gradient is propagated backward through the network using the calculus chain rule. 4. **Weight Updates:** Weights are updated using Gradient Descent to minimize the loss.

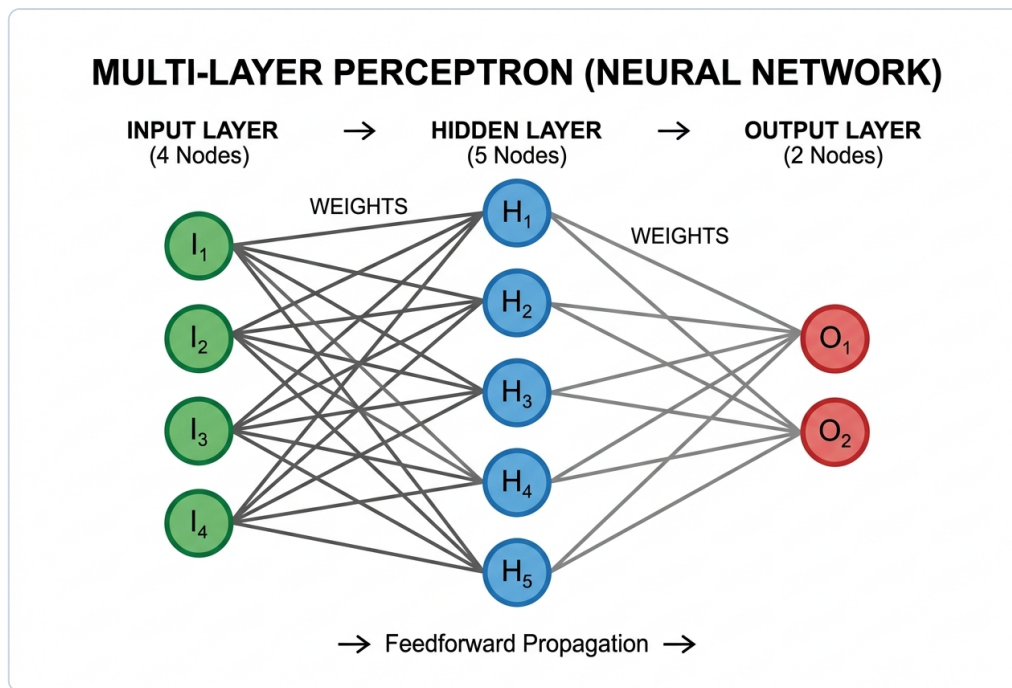


Figure 10.1: Architecture of a Multi-Layer Perceptron (feedforward neural network).

Example Implementation

```
from sklearn.neural_network import MLPClassifier
import numpy as np

# Simple MLP model
X = np.array([[0.0, 0.0], [1.0, 1.0]])
y = np.array([0, 1])

mlp = MLPClassifier(hidden_layer_sizes=(4,), max_iter=1000, random_state=42)
mlp.fit(X, y)
print("Predictions:", mlp.predict(X))
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Import MLPClassifier

Import `MLPClassifier` from `sklearn.neural_network` and instantiate it with `hidden_layer_sizes=(10,)`. Store the classifier in `mlp_easy`.

How to solve: Use: `from sklearn.neural_network import MLPClassifier`, then `mlp_easy = MLPClassifier(hidden_layer_sizes=(10,), random_state=42)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate MLPClassifier
mlp_easy = ____
print(mlp_easy)
```

EASY

Exercise 2: Sigmoid Activation

Write a function `sigmoid(x)` that calculates the standard sigmoid activation function: $\sigma(x) = \frac{1}{1 + e^{-x}}$.

How to solve: Use `1 / (1 + np.exp(-x))`.

TEMPLATE CODE:

```
import numpy as np
def sigmoid(x):
    # TODO: Calculate sigmoid of x
    return ____
print(sigmoid(0.0))
```

EASY

Exercise 3: Single Neuron Prediction

Compute the linear activation of a single neuron: $z = \mathbf{w} \cdot \mathbf{x} + b$. Store the result in z .

How to solve: Use `np.dot(x, w) + b` or `w @ x + b`.

TEMPLATE CODE:

```
x = np.array([2.0, 3.0])
w = np.array([0.5, -1.0])
b = 0.5
# TODO: Calculate w . x + b
z = ____
print(z)
```

EASY

Exercise 4: Import MLPClassifier

Import `MLPClassifier` from `sklearn.neural_network` and instantiate it with

`hidden_layer_sizes=(10,)`. Store the classifier in `mlp_easy`.

How to solve: Import `MLPClassifier` from `sklearn.neural_network`. Train it with `hidden_layer_sizes=(8, 8), activation='relu', and random_state=42`.

TEMPLATE CODE:

```
# TODO: Import and instantiate MLPClassifier
mlp_easy = ____
print(mlp_easy)
```

EASY

Exercise 5: Sigmoid Activation

Write a function `sigmoid(x)` that calculates the standard sigmoid activation function: $\sigma(x) = (1)/(1 + e^{-x})$.

How to solve: Loop over samples. If prediction matches target, do nothing. If they mismatch, update weights: `w += lr * (target - pred) * x`, and update bias: `b += lr * (target - pred)`.

TEMPLATE CODE:

```
import numpy as np
def sigmoid(x):
    # TODO: Calculate sigmoid of x
    return _____
print(sigmoid(0.0))
```

EASY

Exercise 6: Single Neuron Prediction

Compute the linear activation of a single neuron: $z = \mathbf{w} \cdot \mathbf{x} + b$. Store the result in `z`.

How to solve: Implement feedforward: `a1 = sigmoid(np.dot(X, W1) + b1)`, `a2 = sigmoid(np.dot(a1, W2) + b2)`. Backpropagate errors by calculating gradients using derivative of sigmoid: `a * (1 - a)`.

TEMPLATE CODE:

```
x = np.array([2.0, 3.0])
w = np.array([0.5, -1.0])
b = 0.5
# TODO: Calculate w . x + b
z = _____
print(z)
```


CHAPTER 11

Time Series Analysis & Forecasting

Theory & Foundations

Time series data is a sequence of data points indexed in chronological order. Unlike standard datasets, observations in a time series are usually sequentially dependent.

1. Components of Time Series

A time series Y_t is typically decomposed into: - **Trend (T_t)**: The long-term increase or decrease in the data. - **Seasonality (S_t)**: Patterns that repeat at regular intervals (e.g., daily, weekly, monthly). - **Cyclical (C_t)**: Fluctuations that are not of a fixed period (usually economic cycles). - **Irregular/Noise (I_t)**: Random, unpredictable residual fluctuations.

2. Stationarity

A time series is **stationary** if its statistical properties do not change over time. Specifically: 1.

Constant Mean: $E[Y_t] = \mu$ for all t . 2. **Constant Variance**: $\text{Var}(Y_t) = \sigma^2$ for all t . 3. **Constant Autocovariance**: $\text{Cov}(Y_t, Y_{t-k})$ depends only on lag k , not on time t .

Most forecasting models (like ARIMA) assume stationarity. We test stationarity using visual plots (rolling stats) or statistical tests like the **Augmented Dickey-Fuller (ADF) test**. If the series is non-stationary, we apply **differencing**:

$$Y'_t = Y_t - Y_{t-1}$$

3. ARIMA Models

An Autoregressive Integrated Moving Average model is denoted as ARIMA(p, d, q): - **AR (p)**: Autoregressive part uses dependency between an observation and lagged observations. - **I (d)**: Differencing degree required to make the series stationary. - **MA (q)**: Moving Average part models dependency between an observation and residual errors from lagged forecasts.

Deep Dive: Time-Series Dynamics and Autoregressive Fitting

Time series data contains sequential observations where order matters. Unlike standard datasets, time-series observations are often correlated with past values (**autocorrelation**). A time series is

stationary if its statistical properties (mean, variance, covariance) are constant over time. Non-stationary data is hard to model. **Differencing** ($y'_t = y_t - y_{t-1}$) removes trends and seasonality to achieve stationarity, which is verified using the statistical **Augmented Dickey-Fuller (ADF) test**.

ARIMA models stationary time series by combining: - **Autoregression (AR)**: Predicting current values based on past values. - **Integration (I)**: The degree of differencing needed. - **Moving Average (MA)**: Modeling relationship with past forecast errors.

Example Implementation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA

# Generate random walk with trend
np.random.seed(42)
time = pd.date_range(start='2026-01-01', periods=100, freq='D')
value = np.sin(np.linspace(0, 20, 100)) + np.random.normal(scale=0.1, size=100) + np.linspace(0, 5, 100)
df = pd.DataFrame({'value': value}, index=time)

# Visualizing Rolling Statistics
rolling_mean = df['value'].rolling(window=7).mean()
rolling_std = df['value'].rolling(window=7).std()

plt.figure(figsize=(10, 5))
plt.plot(df['value'], label='Original')
plt.plot(rolling_mean, label='Rolling Mean (7d)', color='red')
plt.plot(rolling_std, label='Rolling Std (7d)', color='black')
plt.legend()
plt.title("Rolling Statistics Visualization")
plt.show()

# Perform ADF Test
result = adfuller(df['value'])
print(f"ADF Statistic: {result[0]:.4f}")
print(f"p-value: {result[1]:.4f}")
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Time Series Lag (Shift)

Given a Pandas Series `s`, shift (lag) the values by 1 time step and store the result in `s_lag`.

How to solve: Use `s.shift(1)`.

TEMPLATE CODE:

```
import pandas as pd
s = pd.Series([10, 20, 30, 40, 50])
# TODO: Shift values by 1 step
s_lag = ____
print(s_lag)
```

EASY

Exercise 2: Calculate First Differencing

Apply first-order differencing to the Pandas Series `s`. Store the output in `s_diff`.

How to solve: Use `s.diff()`.

TEMPLATE CODE:

```
s = pd.Series([1, 3, 6, 10, 15])
# TODO: Difference the series
s_diff = ____
print(s_diff)
```

EASY

Exercise 3: Time Series Rolling Mean

Given a Pandas Series `s`, calculate the 3-step rolling mean. Store the result in `s_roll`.

How to solve: Use `s.rolling(window=3).mean()`.

TEMPLATE CODE:

```
s = pd.Series([1.0, 2.0, 3.0, 4.0, 5.0])
# TODO: Calculate rolling mean with window=3
s_roll = ____
print(s_roll)
```

EASY

Exercise 4: Time Series Lag (Shift)

Given a Pandas Series `s`, shift (lag) the values by 1 time step and store the result in `s_lag`.

How to solve: Call `.rolling(window=12)` on the series. Use `.mean()` to get rolling means, and `.std()` to get rolling standard deviations.

TEMPLATE CODE:

```
import pandas as pd
s = pd.Series([10, 20, 30, 40, 50])
# TODO: Shift values by 1 step
s_lag = ____
print(s_lag)
```

EASY

Exercise 5: Calculate First Differencing

Apply first-order differencing to the Pandas Series `s`. Store the output in `s_diff`.

How to solve: Subtract the shifted series to difference: `diff_series = series.diff().dropna()`. Run the ADF test using `adfuller(diff_series)` and inspect the p-value.

TEMPLATE CODE:

```
s = pd.Series([1, 3, 6, 10, 15])
# TODO: Difference the series
s_diff = ____
print(s_diff)
```

EASY

Exercise 6: Time Series Rolling Mean

Given a Pandas Series `s`, calculate the 3-step rolling mean. Store the result in `s_roll`.

How to solve: Fit the `ARIMA(series, order=(1,1,1))` model. Generate forecasts, and calculate MAPE: `np.mean(np.abs((y_true - y_pred) / y_true)) * 100`.

TEMPLATE CODE:

```
s = pd.Series([1.0, 2.0, 3.0, 4.0, 5.0])
# TODO: Calculate rolling mean with window=3
s_roll = ____
print(s_roll)
```

CHAPTER 12

Natural Language Processing (NLP)

Theory & Foundations

Natural Language Processing focuses on the interaction between computers and human languages. Computers require numerical matrices, so converting raw text into numbers is key.

1. Text Preprocessing

Before vectorization, we clean and standardize text: - **Lowercasing**: Converting all text to lowercase. - **Punctuation & Number Removal**: Removing symbols that do not carry semantic meaning. - **Stopword Removal**: Eliminating common words (e.g. 'the', 'is', 'and') that add noise. - **Tokenization**: Splitting sentences into individual words or tokens. - **Stemming/Lemmatization**: Reducing words to their root form (e.g., 'running' → 'run').

2. Bag-of-Words vs. TF-IDF Vectorization

- **Bag-of-Words (BoW)**: Counts occurrence of each word in a document. Ignores grammar and word order.
- **TF-IDF (Term Frequency-Inverse Document Frequency)**: Scales word count by how unique it is across all documents:

$$TF(t, d) = (\text{Count of word } t \text{ in document } d) / (\text{Total words in } d)$$

$$IDF(t) = \log((\text{Total documents } N) / (\text{Documents containing word } t))$$

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

3. Word Embeddings

Represent words as dense vectors in a continuous space, capturing semantic similarities (e.g. Word2Vec, GloVe, FastText). Words with similar meanings end up close to each other.

Deep Dive: Text Vectorization and Semantic Embeddings

Natural Language Processing (NLP) translates unstructured text into numeric matrices: - **Bag of Words (BoW)**: Counts word frequencies, ignoring syntax and word order. - **TF-IDF**: Weighs word frequency against how common the word is across the corpus:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log((N)/(DF(t)))$$

This highlights unique, informative words. - **Cosine Similarity**: Measures the angle between document vectors to assess similarity:

$$\text{similarity} = \cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) / (||\mathbf{A}|| \ ||\mathbf{B}||)$$

- **Word Embeddings**: Map words to dense vectors in a continuous vector space where words with similar meanings are close together, capturing semantic relationships.

Example Implementation

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# TF-IDF Demonstration
docs = ["I love machine learning.", "Machine learning is powerful."]
vec = TfidfVectorizer()
tfidf = vec.fit_transform(docs)

# Output representation
print("Vocabulary:", vec.vocabulary_)
print("TF-IDF Matrix Shape:", tfidf.shape)

# Cosine Similarity
sim = cosine_similarity(tfidf[0], tfidf[1])
print(f"Similarity between document 0 and 1: {sim[0][0]:.4f}")
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Tokenize String

Convert string `text` to lowercase and tokenize it by splitting on spaces. Store the resulting list of words in `words`.

How to solve: Use `text.lower().split()`.

TEMPLATE CODE:

```
text = "Machine Learning is Fun"
# TODO: Lowercase and split text
words = ____
print(words)
```

EASY

Exercise 2: Import CountVectorizer

Import `CountVectorizer` from `sklearn.feature_extraction.text` and instantiate it. Store it in a variable named `vectorizer`.

How to solve: Use: `from sklearn.feature_extraction.text import CountVectorizer`, then `vectorizer = CountVectorizer()`.

TEMPLATE CODE:

```
# TODO: Import and instantiate CountVectorizer
vectorizer = ____
print(vectorizer)
```


EASY

Exercise 3: Dot Product Similarity

Compute the dot product similarity of two 1D NumPy vectors `v1` and `v2`. Store the output in `similarity`.

How to solve: Use `np.dot(v1, v2)` or `v1 @ v2`.

TEMPLATE CODE:

```
import numpy as np
v1 = np.array([1, 0, 1])
v2 = np.array([1, 1, 0])
# TODO: Calculate dot product of v1 and v2
similarity = ____
print(similarity)
```

EASY

Exercise 4: Tokenize String

Convert string `text` to lowercase and tokenize it by splitting on spaces. Store the resulting list of words in `words`.

How to solve: Convert text to lowercase, strip punctuation using a regular expression (e.g. `re.sub(r'^a-z0-9\s', '', text)`), split into tokens, and filter out tokens present in the stopword list.

TEMPLATE CODE:

```
text = "Machine Learning is Fun"
# TODO: Lowercase and split text
words = ____
print(words)
```

EASY

Exercise 5: Import CountVectorizer

Import `CountVectorizer` from `sklearn.feature_extraction.text` and instantiate it. Store it in a variable named `vectorizer`.

How to solve: Fit `TfidfVectorizer` on the preprocessed corpus. Compute the similarity matrix by calling `cosine_similarity(tfidf_matrix)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate CountVectorizer
vectorizer = _____
print(vectorizer)
```

EASY

Exercise 6: Dot Product Similarity

Compute the dot product similarity of two 1D NumPy vectors `v1` and `v2`. Store the output in `similarity`.

How to solve: For each document, average the embedding vectors of all words in the document. Train a `LogisticRegression` model on these document vectors.

TEMPLATE CODE:

```
import numpy as np
v1 = np.array([1, 0, 1])
v2 = np.array([1, 1, 0])
# TODO: Calculate dot product of v1 and v2
similarity = _____
print(similarity)
```

CHAPTER 13

Anomaly Detection

Theory & Foundations

Anomaly detection is the identification of rare items, events, or observations which raise suspicions by differing significantly from the majority of the data.

1. Statistical Outlier Detection

- **Z-Score**: Measures how many standard deviations a data point is from the mean. Outliers are typically identified if:

$$|Z| > 3 \text{ where } Z = (x - \mu)/(\sigma)$$

- **Interquartile Range (IQR)**: Calculates the spread of the middle 50% of the data. Outliers lie outside the bounds:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR}$$

$$\text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}$$

Where $\text{IQR} = Q_3 - Q_1$, and Q_1, Q_3 are the 25th and 75th percentiles.

2. Isolation Forest

An unsupervised tree-based algorithm. Anomalies are isolated closer to the root of the trees because they require fewer splits to partition from the rest of the dataset.

3. Mahalanobis Distance

For multidimensional data, Mahalanobis distance accounts for the correlation between features:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Where $\boldsymbol{\mu}$ is the mean vector and $\boldsymbol{\Sigma}$ is the covariance matrix.

Deep Dive: Outlier Boundaries and Distance Metrics

Anomalies are rare observations that deviate significantly from standard distributions: - **Statistical Thresholds**: Z-score marks points beyond a standard deviation limit ($|Z| > 3$). IQR defines outliers beyond $[Q1 - 1.5IQR, Q3 + 1.5IQR]$. - **Isolation Forest**: An unsupervised algorithm that isolates anomalies by partitioning features. Since anomalies have extreme values, they require fewer splits to isolate, resulting in shorter paths in decision trees. - **Mahalanobis Distance**: Measures distance between a point and a distribution in multi-dimensional space, accounting for correlations between variables:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

where $\boldsymbol{\Sigma}^{-1}$ is the inverse covariance matrix.

Example Implementation

```
import numpy as np
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt

# Generate normal data
np.random.seed(42)
X = np.random.normal(loc=0.0, scale=1.0, size=(100, 2))
# Add anomaly outlier
X = np.vstack([X, [4.0, 4.0]])

# Fit Isolation Forest
clf = IsolationForest(contamination=0.01, random_state=42)
preds = clf.fit_predict(X) # 1 = normal, -1 = anomaly

print("Outlier prediction count:", np.sum(preds == -1))
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Calculate Mean and Std

Given a 1D NumPy array `arr`, calculate its mean and standard deviation. Store them in `mean_val` and `std_val`.

How to solve: Use `np.mean(arr)` and `np.std(arr)`.

TEMPLATE CODE:

```
import numpy as np
arr = np.array([2, 4, 4, 4, 5, 5, 7, 9])
# TODO: Compute mean and std
mean_val = ____
std_val = ____
print(f"Mean: {mean_val}, Std: {std_val}")
```

EASY

Exercise 2: Identify Outliers via Z-score

Given an array of Z-scores `z_scores`, identify indices where the absolute Z-score is strictly greater than 2.0. Store these indices in `outlier_indices`.

How to solve: Use `np.where(np.abs(z_scores) > 2.0)[0]`.

TEMPLATE CODE:

```
z_scores = np.array([-0.5, 2.5, 0.1, -2.1, 1.2])
# TODO: Find indices where absolute Z-score > 2.0
outlier_indices = ____
print(outlier_indices)
```

EASY

Exercise 3: Import Isolation Forest

Import `IsolationForest` from `sklearn.ensemble` and instantiate it setting `random_state=42`. Store the model in `iso_forest`.

How to solve: Use: `from sklearn.ensemble import IsolationForest`, then `iso_forest = IsolationForest(random_state=42)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate IsolationForest
iso_forest = ____
print(iso_forest)
```

EASY

Exercise 4: Calculate Mean and Std

Given a 1D NumPy array `arr`, calculate its mean and standard deviation. Store them in `mean_val` and `std_val`.

How to solve: Z-scores are calculated as $(x - \text{mean}) / \text{std}$. IQR is $Q3 - Q1$, and outliers are points outside the range $[Q1 - 1.5 * \text{IQR}, Q3 + 1.5 * \text{IQR}]$.

TEMPLATE CODE:

```
import numpy as np
arr = np.array([2, 4, 4, 4, 5, 5, 7, 9])
# TODO: Compute mean and std
mean_val = ____
std_val = ____
print(f"Mean: {mean_val}, Std: {std_val}")
```

EASY

Exercise 5: Identify Outliers via Z-score

Given an array of Z-scores `z_scores`, identify indices where the absolute Z-score is strictly greater than `2.0`. Store these indices in `outlier_indices`.

How to solve: Fit `IsolationForest(contamination=0.05, random_state=42)`. Call `.predict(X)`, where anomalies are labeled as `-1`.

TEMPLATE CODE:

```
z_scores = np.array([-0.5, 2.5, 0.1, -2.1, 1.2])
# TODO: Find indices where absolute Z-score > 2.0
outlier_indices = ____
print(outlier_indices)
```

EASY

Exercise 6: Import Isolation Forest

Import `IsolationForest` from `sklearn.ensemble` and instantiate it setting `random_state=42`. Store the model in `iso_forest`.

How to solve: Calculate the inverse covariance matrix using `np.linalg.inv(np.cov(X.T))`. Calculate Mahalanobis distances using the formula: `d = np.sqrt(np.sum(diff @ inv_cov * diff, axis=1))`.

TEMPLATE CODE:

```
# TODO: Import and instantiate IsolationForest
iso_forest = ____
print(iso_forest)
```

CHAPTER 14

Ensemble Learning & Gradient Boosting

Theory & Foundations

Ensemble learning combines predictions from multiple models to generate a single, more robust prediction.

1. Types of Ensembles

- **Voting/Averaging**: Combines predictions from different model classes (e.g. SVM + Decision Tree + Logistic Regression).
- *Hard Voting*: Class predictions are decided by majority vote.
- *Soft Voting*: Prediction is based on the argmax of the sum of predicted probabilities.
- **Bagging (Bootstrap Aggregation)**: Trains multiple models of the same class in parallel on different bootstrapped datasets (e.g., Random Forest). Reduces variance.
- **Boosting**: Trains models sequentially. Each new model focuses on correcting the errors (residuals) of the previous models. Reduces bias.
- **Stacking**: Trains base models on the training data, then uses their predictions as features to train a meta-classifier.

2. Boosting Algorithms

- **AdaBoost**: Re-weights samples at each step, putting more emphasis on previously misclassified observations.
- **Gradient Boosting**: Fits new estimators to the negative gradient (pseudo-residuals) of the loss function. Popular optimized implementations: **XGBoost**, **LightGBM**, and **CatBoost**.

Deep Dive: Ensemble Architecture & Boosting Mechanics

Ensemble learning combines multiple models to reduce error: - **Voting Classifiers**: Combine different models. **Hard voting** uses majority vote. **Soft voting** averages predicted probabilities, weighting confident predictions higher. - **Boosting**: Models are trained sequentially. **AdaBoost** assigns higher weights to samples misclassified by previous models. **Gradient Boosting** fits new models to the residual errors (gradients) of the loss function. - **Stacking**: Trains base models on the training set, uses their predictions as features, and trains a meta-model to make the final prediction.

Example Implementation

```
from sklearn.ensemble import VotingClassifier, AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Base models
clf1 = LogisticRegression(random_state=42)
clf2 = DecisionTreeClassifier(max_depth=3, random_state=42)

# Voting ensemble
voting = VotingClassifier(estimators=[('lr', clf1), ('dt', clf2)], voting='soft')
X, y = make_classification(n_samples=100, random_state=42)
voting.fit(X, y)
print("Voting Score:", voting.score(X, y))
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Hard Voting Logic

Given predictions from 3 different models (`pred1`, `pred2`, `pred3`), perform hard majority voting for each sample and store the outputs in `final_preds`.

How to solve: Stack predictions vertically and use scipy's mode or sum them: `final_preds = (pred1 + pred2 + pred3) >= 2`.

TEMPLATE CODE:

```
pred1 = np.array([0, 1, 0])
pred2 = np.array([0, 1, 1])
pred3 = np.array([1, 1, 0])
# TODO: Calculate majority predictions
final_preds = ____
print(final_preds)
```

EASY

Exercise 2: Import AdaBoost Classifier

Import `AdaBoostClassifier` from `sklearn.ensemble` and instantiate it. Store it in a variable named `ada_easy`.

How to solve: Use: `from sklearn.ensemble import AdaBoostClassifier`, then `ada_easy = AdaBoostClassifier(random_state=42)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate AdaBoostClassifier
ada_easy = ____
print(ada_easy)
```

EASY

Exercise 3: Import Bagging Classifier

Import `BaggingClassifier` from `sklearn.ensemble` and instantiate it. Store it in `bagging_easy`.

How to solve: Use: `from sklearn.ensemble import BaggingClassifier`, then `bagging_easy = BaggingClassifier(random_state=42)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate BaggingClassifier
bagging_easy = ____
print(bagging_easy)
```

EASY

Exercise 4: Hard Voting Logic

Given predictions from 3 different models (`pred1`, `pred2`, `pred3`), perform hard majority voting for each sample and store the outputs in `final_preds`.

How to solve: Instantiate `VotingClassifier(estimators, voting='soft')`. Ensure the base estimators support probability output, fit the ensemble, and score on the test set.

TEMPLATE CODE:

```
pred1 = np.array([0, 1, 0])
pred2 = np.array([0, 1, 1])
pred3 = np.array([1, 1, 0])
# TODO: Calculate majority predictions
final_preds = ____
print(final_preds)
```

EASY

Exercise 5: Import AdaBoost Classifier

Import `AdaBoostClassifier` from `sklearn.ensemble` and instantiate it. Store it in a variable named `ada_easy`.

How to solve: Use the `staged_predict()` method of the trained `AdaBoostClassifier`. Loop through stages to calculate and plot the validation error at each boosting iteration.

TEMPLATE CODE:

```
# TODO: Import and instantiate AdaBoostClassifier
ada_easy = ____
print(ada_easy)
```

EASY

Exercise 6: Import Bagging Classifier

Import `BaggingClassifier` from `sklearn.ensemble` and instantiate it. Store it in `bagging_easy`.

How to solve: Train base classifiers. Use their predicted probabilities on the validation set as features to train a meta-classifier (e.g. Logistic Regression).

TEMPLATE CODE:

```
# TODO: Import and instantiate BaggingClassifier
bagging_easy = ____
print(bagging_easy)
```

CHAPTER 15

Dimensionality Reduction & Manifold Learning

Theory & Foundations

In machine learning, high-dimensional data (hundreds of features) can cause models to overfit and makes visualization impossible. This is known as the **Curse of Dimensionality**.

1. Linear Reduction Methods

- **PCA (Principal Component Analysis)**: Unsupervised method maximizing data variance.
- **LDA (Linear Discriminant Analysis)**: Supervised method maximizing class separability:

$$S_W = \sum_c S_c \quad S_B = \sum_c N_c (\mathbf{m}_c - \mathbf{m})(\mathbf{m}_c - \mathbf{m})^T$$

LDA finds a projection that maximizes the ratio of between-class variance to within-class variance (S_B / S_W).

2. Non-linear & Manifold Methods

When data lies on a complex curved manifold, linear projections fail. - **Kernel PCA**: Employs the kernel trick to map data into high-dimensional space where PCA is applied linearly. - **t-SNE (t-Distributed Stochastic Neighbor Embedding)**: Converts Euclidean distances between data points into conditional probabilities representing similarities, preserving local structure for 2D/3D visualizations. - **UMAP (Uniform Manifold Approximation and Projection)**: Similar to t-SNE but faster and preserves more global structure.

Deep Dive: Manifold Mapping and Class Discriminants

High-dimensional datasets suffer from the "curse of dimensionality". Manifold learning finds lower-dimensional projections that preserve the dataset's geometry: - **Linear Discriminant Analysis (LDA)**: A supervised method that projects data onto axes that maximize distance between class means while minimizing variance within classes. - **Kernel PCA**: Uses kernels to project data with non-linear structures. - **t-Distributed Stochastic Neighbor Embedding (t-SNE)**: A non-linear technique that maps high-dimensional datasets to 2D or 3D, keeping similar points close and dissimilar points far apart, ideal for visualizing clusters.

Example Implementation

```
from sklearn.datasets import make_circles
from sklearn.decomposition import KernelPCA
import matplotlib.pyplot as plt

# Generate non-linear concentric circles
X, y = make_circles(n_samples=100, factor=0.5, noise=0.05, random_state=42)

# Fit Kernel PCA
kpca = KernelPCA(n_components=2, kernel='rbf', gamma=15)
X_kpca = kpca.fit_transform(X)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(X[:, 0], X[:, 1], c=y, cmap='viridis')
axes[0].set_title("Original Data Space")
axes[1].scatter(X_kpca[:, 0], X_kpca[:, 1], c=y, cmap='viridis')
axes[1].set_title("Kernel PCA Space")
plt.show()
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Import LDA

Import `LinearDiscriminantAnalysis` from `sklearn.discriminant_analysis` and instantiate it. Store it in a variable named `lda_easy`.

How to solve: Use: `from sklearn.discriminant_analysis import LinearDiscriminantAnalysis`, then `lda_easy = LinearDiscriminantAnalysis()`.

TEMPLATE CODE:

```
# TODO: Import and instantiate LDA
lda_easy = ____
print(lda_easy)
```

EASY

Exercise 2: Import t-SNE

Import `TSNE` from `sklearn.manifold` and instantiate it setting `n_components=2`. Store the instance in `tsne_easy`.

How to solve: Use: `from sklearn.manifold import TSNE`, then `tsne_easy = TSNE(n_components=2, random_state=42)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate TSNE
tsne_easy = ____
print(tsne_easy)
```

EASY

Exercise 3: Silhouette Score Calculation

Import `silhouette_score` from `sklearn.metrics` and calculate the silhouette score for cluster assignments `labels` on dataset `x`.

How to solve: Call `silhouette_score(X, labels)`.

TEMPLATE CODE:

```
from sklearn.metrics import silhouette_score
X = np.array([[1, 2], [1, 3], [10, 12], [10, 13]])
labels = np.array([0, 0, 1, 1])
# TODO: Calculate silhouette score
score = ____
print(score)
```

EASY

Exercise 4: Import LDA

Import `LinearDiscriminantAnalysis` from `sklearn.discriminant_analysis` and instantiate it. Store it in a variable named `lda_easy`.

How to solve: Import `LinearDiscriminantAnalysis`. Train it using `lda.fit(X_train, y_train)`, then project features using `lda.transform(X_train)`.

TEMPLATE CODE:

```
# TODO: Import and instantiate LDA
lda_easy = ____
print(lda_easy)
```


EASY

Exercise 5: Import t-SNE

Import `TSNE` from `sklearn.manifold` and instantiate it setting `n_components=2`. Store the instance in `tsne_easy`.

How to solve: Use `KernelPCA(n_components=2, kernel='rbf', gamma=15)` to project circular, non-linear data into a linearly separable 2D space.

TEMPLATE CODE:

```
# TODO: Import and instantiate TSNE
tsne_easy = ____
print(tsne_easy)
```

EASY

Exercise 6: Silhouette Score Calculation

Import `silhouette_score` from `sklearn.metrics` and calculate the silhouette score for cluster assignments `labels` on dataset `X`.

How to solve: Project data using both PCA and t-SNE. Calculate and compare cluster quality using `silhouette_score(projection, labels)`.

TEMPLATE CODE:

```
from sklearn.metrics import silhouette_score
X = np.array([[1, 2], [1, 3], [10, 12], [10, 13]])
labels = np.array([0, 0, 1, 1])
# TODO: Calculate silhouette score
score = ____
print(score)
```

CHAPTER 16

Recommender Systems

Theory & Foundations

Recommender systems aim to predict the rating or preference a user would give to an item, helping users discover new items (books, movies, products).

1. Content-Based Filtering

Recommends items similar to those a user liked in the past. - **Item Profile**: Represents items as vectors of features (e.g. genres, keywords). - **User Profile**: Represents user preferences as a vector in the same feature space. - **Scoring**: Predicted preference is calculated using the dot product or cosine similarity between user and item profiles:

$$\text{Score}(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v}$$

2. Collaborative Filtering (CF)

Recommends items based on the behavior of other similar users. - **User-Based CF**: Find users with similar rating histories. The rating of user u for item i is calculated as a similarity-weighted average of other users' ratings:

$$\hat{r}_{u,i} = \frac{\sum_{v \in N} \text{sim}(u, v) \cdot r_{v,i}}{\sum_{v \in N} |\text{sim}(u, v)|}$$

- **Item-Based CF**: Find items similar to item i that user u has previously rated.

3. Latent Factor Models & Matrix Factorization

Decomposes the sparse rating matrix R (dimension $M \times N$) into two lower-rank dense matrices P ($M \times K$, user latent factors) and Q ($N \times K$, item latent factors):

$$R \approx P \cdot Q^T$$

We learn P and Q by minimizing the regularized reconstruction error on observed ratings:

$$\min_{P, Q} \sum_{(u, i) \in K} (R_{u, i} - P_u \cdot Q_i^T)^2 + \lambda (\|P_u\|^2 + \|Q_i\|^2)$$

Deep Dive: Latent Factors and Collaborative Filtering

Recommender systems connect users with relevant content: - **Content-Based Filtering:**

Recommends items similar to those a user previously interacted with, based on item features. -

Collaborative Filtering: Recommends items by matching users with similar tastes (User-Based) or identifying items commonly liked together (Item-Based). - **Matrix Factorization:** Decomposes the

user-item rating matrix R into user latent factors P and item latent factors Q , such that $R \approx P \cdot Q^T$.

These factors are learned iteratively using Stochastic Gradient Descent (SGD) to minimize prediction error.

Example Implementation

```
import numpy as np
import pandas as pd

# Collaborative Filtering ratings matrix example
# Rows = Users, Cols = Items (0 indicates unrated)
ratings = np.array([
    [5, 3, 0, 1],
    [4, 0, 0, 1],
    [1, 1, 0, 5],
    [1, 0, 0, 4],
    [0, 1, 5, 4],
])
print("Ratings Matrix Shape:", ratings.shape)
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: User-Item Dot Product

Given a 1D user preference vector u and a 1D item features vector v , calculate the recommendation score (dot product). Store it in `score`.

How to solve: Use `np.dot(u, v)` or `u @ v`.

TEMPLATE CODE:

```
u = np.array([5.0, 1.0])
v = np.array([4.0, 2.0])
# TODO: Compute dot product score
score = ____
print(score)
```

EASY

Exercise 2: Vector Magnitude (Norm)

Calculate the L2 Euclidean norm (magnitude) of a 1D NumPy array v . Store it in `magnitude`.

How to solve: Use `np.linalg.norm(v)`.

TEMPLATE CODE:

```
v = np.array([3.0, 4.0])
# TODO: Calculate vector magnitude using NumPy
magnitude = ____
print(magnitude)
```

EASY

Exercise 3: Pivot Dataframe

Pivot the DataFrame `df` so that rows correspond to 'User', columns correspond to 'Item', and values correspond to 'Rating'. Store the result in `utility_matrix`.

How to solve: Use `df.pivot(index='User', columns='Item', values='Rating')`.

TEMPLATE CODE:

```
df = pd.DataFrame({'User': ['A', 'A', 'B'], 'Item': ['I1', 'I2', 'I1'], 'Rating': [5, 4, 3]})
# TODO: Pivot to create utility matrix
utility_matrix = ____
print(utility_matrix)
```

EASY

Exercise 4: User-Item Dot Product

Given a 1D user preference vector `u` and a 1D item features vector `v`, calculate the recommendation score (dot product). Store it in `score`.

How to solve: Compute similarity scores: `scores = item_features @ user_profile`. Sort the scores in descending order using `np.argsort(scores)[-1]` to get recommendations.

TEMPLATE CODE:

```
u = np.array([5.0, 1.0])
v = np.array([4.0, 2.0])
# TODO: Compute dot product score
score = ____
print(score)
```

EASY

Exercise 5: Vector Magnitude (Norm)

Calculate the L2 Euclidean norm (magnitude) of a 1D NumPy array `v`. Store it in `magnitude`.

How to solve: Calculate cosine similarities between User 0 and other users. Use the similarities as weights to compute the predicted rating for User 0 on Item 2.

TEMPLATE CODE:

```
v = np.array([3.0, 4.0])
# TODO: Calculate vector magnitude using NumPy
magnitude = ____
print(magnitude)
```

EASY

Exercise 6: Pivot Dataframe

Pivot the DataFrame `df` so that rows correspond to 'User', columns correspond to 'Item', and values correspond to 'Rating'. Store the result in `utility_matrix`.

How to solve: Implement the SGD loops: for active ratings, calculate error: `error = R[u,i] - np.dot(P[u], Q[i])`. Update: `P[u] += alpha * (2 * error * Q[i] - beta * P[u])`.

TEMPLATE CODE:

```
df = pd.DataFrame({'User': ['A', 'A', 'B'], 'Item': ['I1', 'I2', 'I1'], 'Rating': [5, 4, 3]})
# TODO: Pivot to create utility matrix
utility_matrix = ____
print(utility_matrix)
```

CHAPTER 17

Association Rule Learning

Theory & Foundations

Association rule learning is a rule-based machine learning method for discovering interesting relations (co-occurrences) between variables in large databases. It is widely used in **Market Basket Analysis** to identify products frequently bought together.

1. Metrics for Rule Evaluation

Let A and B be sets of items (itemsets). A rule is written as $A \rightarrow B$ (e.g. $\{\text{milk}\} \rightarrow \{\text{bread}\}$). -

Support: Measures how frequently the itemset appears in the database:

$$\text{Support}(A) = (\text{Count}(A)) / (\text{Total Transactions } N)$$

- **Confidence:** Measures how often the rule is found to be true. The conditional probability of buying B given A:

$$\text{Confidence}(A \rightarrow B) = (\text{Support}(A \cup B)) / (\text{Support}(A))$$

- **Lift:** Measures how much more often A and B occur together than expected if they were statistically independent. A lift > 1 indicates positive correlation:

$$\text{Lift}(A \rightarrow B) = (\text{Confidence}(A \rightarrow B)) / (\text{Support}(B))$$

2. Apriori Algorithm

Finding frequent itemsets by checking all combinations is computationally expensive. Apriori uses the **downward closure property**:

If an itemset is frequent, all of its subsets must also be frequent. It generates candidates of size k by joining frequent itemsets of size $k-1$, pruning candidates containing non-frequent subsets.

Deep Dive: Market Basket Analysis and Association Rules

Association Rule Learning discovers relationships between items in transaction data: - **Support**: Frequency of an itemset:

$$\text{Support}(A \rightarrow B) = (\text{Frequency}(A \cup B)) / (\text{Total Transactions})$$

- **Confidence**: Accuracy of the rule:

$$\text{Confidence}(A \rightarrow B) = (\text{Support}(A \cup B)) / (\text{Support}(A))$$

- **Lift**: Strength of the rule compared to random chance:

$$\text{Lift}(A \rightarrow B) = (\text{Confidence}(A \rightarrow B)) / (\text{Support}(B))$$

A Lift > 1 indicates that items are frequently bought together. - **Apriori Algorithm**: Reduces search space by pruning itemsets whose subsets are infrequent.

Example Implementation

```
# Simple transactions dataset
transactions = [
    ['milk', 'bread'],
    ['milk', 'diaper', 'beer'],
    ['milk', 'bread', 'diaper', 'beer'],
    ['bread', 'diaper', 'beer'],
    ['milk', 'bread', 'diaper']
]
total_transactions = len(transactions)
print("Total Transactions:", total_transactions)
```


Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Calculate Support

Given that a product is bought in 30 transactions out of a total of 120 transactions, calculate the support of the product. Store it in `support`.

How to solve: Use `count / total`.

TEMPLATE CODE:

```
count = 30
total = 120
# TODO: Calculate support
support = ____
print(support)
```

EASY

Exercise 2: Calculate Confidence

If items A and B are bought together in 15 transactions, and item A is bought alone in 20 transactions, calculate the confidence of the association rule $A \rightarrow B$. Store it in `confidence`.

How to solve: Use `count_both / count_A`.

TEMPLATE CODE:

```
count_both = 15
count_A = 20
# TODO: Calculate confidence
confidence = ____
print(confidence)
```

EASY

Exercise 3: Calculate Lift

Calculate the lift of a rule $A \rightarrow B$, given confidence is `0.75` and support of item B is `0.5`. Store it in `lift`.

How to solve: Use confidence / support_B.

TEMPLATE CODE:

```
confidence = 0.75
support_B = 0.5
# TODO: Calculate lift
lift = ____
print(lift)
```

EASY

Exercise 4: Calculate Support

Given that a product is bought in `30` transactions out of a total of `120` transactions, calculate the support of the product. Store it in `support`.

How to solve: Support is transactions with both items divided by total. Confidence is both items divided by antecedent count. Lift is confidence divided by consequent support.

TEMPLATE CODE:

```
count = 30
total = 120
# TODO: Calculate support
support = ____
print(support)
```

EASY

Exercise 5: Calculate Confidence

If items A and B are bought together in 15 transactions, and item A is bought alone in 20 transactions, calculate the confidence of the association rule $A \rightarrow B$. Store it in `confidence`.

How to solve: Join frequent itemsets of size k to generate candidates of size $k+1$. Filter candidates by counting their occurrences in transactions and checking if they meet min support.

TEMPLATE CODE:

```
count_both = 15
count_A = 20
# TODO: Calculate confidence
confidence = ____
print(confidence)
```

EASY

Exercise 6: Calculate Lift

Calculate the lift of a rule $A \rightarrow B$, given confidence is 0.75 and support of item B is 0.5. Store it in `lift`.

How to solve: For each frequent itemset, generate possible rules, calculate confidence and lift, and filter by thresholds.

TEMPLATE CODE:

```
confidence = 0.75
support_B = 0.5
# TODO: Calculate lift
lift = ____
print(lift)
```

CHAPTER 18

Semi-Supervised Learning

Theory & Foundations

Semi-supervised learning is a branch of machine learning that combines a small amount of labeled data with a large amount of unlabeled data during training. This is extremely useful because labeling data can be expensive and time-consuming, while unlabeled data is often cheap and abundant.

1. Label Propagation and Label Spreading

Graph-based algorithms that model the data points as nodes in a connected graph. - **Edges:** Weighted by similarity between points. - **Propagation:** Labels propagate from labeled nodes along the edges to unlabeled nodes. - **Label Spreading:** A variant that uses a normalized Laplacian matrix and adds a regularization term to make it robust to noise.

2. Pseudo-Labeling (Self-Training)

An iterative wrapper-method: 1. Train a classifier on the small set of labeled data. 2. Predict class probabilities for the unlabeled data. 3. Select unlabeled samples whose predicted probability exceeds a high threshold (e.g. ≥ 0.9). 4. Add these samples to the training set as labeled, using their predicted label ("pseudo-label"). 5. Retrain the classifier on the expanded training set and repeat.

Deep Dive: Graph Label Propagation and Self-Training

Semi-Supervised learning utilizes a small labeled dataset and a large unlabeled dataset: - **Label Propagation:** Builds a graph where data points are nodes and edges represent similarity. Labels are propagated from labeled nodes to unlabeled neighbors based on edge weights. - **Self-Training (Pseudo-Labeling):** Trains a model on labeled data, predicts probabilities for unlabeled data, adds predictions that exceed a confidence threshold to the training set, and retrains the model. - **Label Spreading:** A variant of Label Propagation that uses a normalized Laplacian matrix and adds regularization to prevent noise propagation.

Example Implementation

```
from sklearn.semi_supervised import LabelPropagation
from sklearn.datasets import make_classification
import numpy as np

# Make data where -1 denotes unlabeled points
X, y = make_classification(n_samples=100, n_features=4, n_classes=2,
                           random_state=42)
y_masked = y.copy()
y_masked[80:] = -1 # Mask 20% labels as unlabeled

lp = LabelPropagation().fit(X, y_masked)
print("Transduced Labels for unlabeled points:", lp.transduction_[80:])
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Import Label Propagation

Import `LabelPropagation` from `sklearn.semi_supervised` and instantiate it. Store it in a variable named `lp_easy`.

How to solve: Use: `from sklearn.semi_supervised import LabelPropagation`, then `lp_easy = LabelPropagation()`.

TEMPLATE CODE:

```
# TODO: Import and instantiate LabelPropagation
lp_easy = ____
print(lp_easy)
```

EASY

Exercise 2: Count Unlabeled Samples

In semi-supervised learning, unlabeled samples are represented by a label of `-1`. Given labels `y_masked`, count the number of unlabeled samples. Store it in `unlabeled_count`.

How to solve: Use `np.sum(y_masked == -1)`.

TEMPLATE CODE:

```
y_masked = np.array([0, 1, -1, 0, -1, 1, -1])
# TODO: Count how many labels are equal to -1
unlabeled_count = ____
print(unlabeled_count)
```

EASY

Exercise 3: Import Label Spreading

Import `LabelSpreading` from `sklearn.semi_supervised` and instantiate it. Store it in a variable named `ls_easy`.

How to solve: Use: `from sklearn.semi_supervised import LabelSpreading`, then `ls_easy = LabelSpreading()`.

TEMPLATE CODE:

```
# TODO: Import and instantiate LabelSpreading
ls_easy = ____
print(ls_easy)
```

EASY

Exercise 4: Import Label Propagation

Import `LabelPropagation` from `sklearn.semi_supervised` and instantiate it. Store it in a variable named `lp_easy`.

How to solve: Train `LabelPropagation(kernel='knn', n_neighbors=5)`. Evaluate accuracy on the unlabeled indices by comparing model predictions with true labels.

TEMPLATE CODE:

```
# TODO: Import and instantiate LabelPropagation
lp_easy = ____
print(lp_easy)
```

EASY

Exercise 5: Count Unlabeled Samples

In semi-supervised learning, unlabeled samples are represented by a label of `-1`. Given labels `y_masked`, count the number of unlabeled samples. Store it in `unlabeled_count`.

How to solve: Fit classifier on labeled data. Predict probabilities on unlabeled data. Find samples exceeding the confidence threshold, add them to the training set, and repeat.

TEMPLATE CODE:

```
y_masked = np.array([0, 1, -1, 0, -1, 1, -1])
# TODO: Count how many labels are equal to -1
unlabeled_count = ____
print(unlabeled_count)
```

EASY

Exercise 6: Import Label Spreading

Import `LabelSpreading` from `sklearn.semi_supervised` and instantiate it. Store it in a variable named `ls_easy`.

How to solve: Perform a grid search over gamma values. Train `LabelSpreading(kernel='rbf', gamma=g)` for each value and select the gamma that yields the highest test accuracy.

TEMPLATE CODE:

```
# TODO: Import and instantiate LabelSpreading
ls_easy = ____
print(ls_easy)
```


CHAPTER 19

Classification on Imbalanced Data

Theory & Foundations

Class imbalance occurs when the classes in a classification dataset are not represented equally. For example, in fraud detection, only 0.1% of transactions may be fraudulent.

1. The Accuracy Metric Trap

When classes are highly imbalanced, **accuracy** is a misleading metric. If 99% of samples are of class 0, a naive model predicting 0 for all samples achieves 99% accuracy but fails completely. We must evaluate models using: - **Precision**: $(TP)/(TP + FP)$ (Fraction of predicted positives that are correct). - **Recall**: $(TP)/(TP + FN)$ (Fraction of actual positives found). - **F1 Score**: $2 \cdot (Precision \cdot Recall)/(Precision + Recall)$ (Harmonic mean). - **Macro F1 Score**: Averaging the F1 scores calculated separately for each class.

2. Resampling Techniques

- **Oversampling**: Duplicating minority class samples or synthesizing new ones (**SMOTE**).
- **Undersampling**: Removing majority class samples to balance frequencies.

3. Cost-Sensitive Learning

Instead of balancing the datasets, we modify the loss function during training, assigning a higher weight (penalty) to misclassifications of the minority class. In scikit-learn, this is set using the parameter `class_weight='balanced'`.

Deep Dive: Imbalanced Distribution Remediation

Standard machine learning models assume balanced class distributions. With highly imbalanced data (e.g., fraud detection), a model can achieve 99% accuracy by predicting the majority class. Remedies include: - **Evaluation Metrics**: Evaluate macro F1-score, precision, recall, or Area Under ROC curve instead of accuracy. - **Oversampling**: Replicates minority class instances (e.g., Random Oversampling, SMOTE) to balance the distribution. - **Cost-Sensitive Learning**: Penalizes misclassifying the minority class more heavily by adjusting loss function weights (e.g., setting `class_weight='balanced'`).

Example Implementation

```
from sklearn.datasets import make_classification
from sklearn.metrics import classification_report
from sklearn.svm import SVC
import numpy as np

# Highly imbalanced dataset
X, y = make_classification(n_samples=1000, weights=[0.95, 0.05], random_state=42)
print("Class distribution:", np.bincount(y))

# Train classifier
clf = SVC(kernel='linear', random_state=42).fit(X, y)
print("Classification Report:")
print(classification_report(y, clf.predict(X)))
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Get Class Distribution

Given a target vector `y`, find the unique class labels and their corresponding counts. Store the counts in a variable named `counts`.

How to solve: Use `np.unique(y, return_counts=True)`.

TEMPLATE CODE:

```
y = np.array([0, 0, 0, 0, 0, 0, 0, 0, 0, 1])
# TODO: Find class counts
classes, counts = ____
print("Counts:", counts)
```

EASY

Exercise 2: Balanced Weight Decision Tree

Instantiate a `DecisionTreeClassifier` with `class_weight='balanced'`. Store it in `dt_balanced`.

How to solve: Use `DecisionTreeClassifier(class_weight='balanced', random_state=42)`.

TEMPLATE CODE:

```
from sklearn.tree import DecisionTreeClassifier
# TODO: Instantiate decision tree with balanced class weights
dt_balanced = ____
print(dt_balanced.class_weight)
```

EASY

Exercise 3: Calculate Balanced Accuracy

Import `balanced_accuracy_score` from `sklearn.metrics` and calculate it on the given labels.

How to solve: Use `balanced_accuracy_score(y_true, y_pred)`.

TEMPLATE CODE:

```
# TODO: Import balanced_accuracy_score and calculate
y_true = np.array([0, 0, 0, 1])
y_pred = np.array([0, 0, 0, 0])
bal_acc = ____
print(bal_acc)
```

EASY

Exercise 4: Get Class Distribution

Given a target vector `y`, find the unique class labels and their corresponding counts. Store the counts in a variable named `counts`.

How to solve: Calculate the classification accuracy and the macro-averaged F1-score: `f1_score(y_true, y_pred, average='macro')`. Note how accuracy remains high while F1 drops.

TEMPLATE CODE:

```
y = np.array([0, 0, 0, 0, 0, 0, 0, 0, 0, 1])
# TODO: Find class counts
classes, counts = ____
print("Counts:", counts)
```

EASY

Exercise 5: Balanced Weight Decision Tree

Instantiate a `DecisionTreeClassifier` with `class_weight='balanced'`. Store it in `dt_balanced`.

How to solve: Separate majority and minority indices. Sample minority indices with replacement using `np.random.choice()` to match the majority size. Concatenate indices.

TEMPLATE CODE:

```
from sklearn.tree import DecisionTreeClassifier
# TODO: Instantiate decision tree with balanced class weights
dt_balanced = ____
print(dt_balanced.class_weight)
```

EASY

Exercise 6: Calculate Balanced Accuracy

Import `balanced_accuracy_score` from `sklearn.metrics` and calculate it on the given labels.

How to solve: Train an SVM classifier, setting the hyperparameter `class_weight='balanced'` to penalize minority misclassifications more heavily.

TEMPLATE CODE:

```
# TODO: Import balanced_accuracy_score and calculate
y_true = np.array([0, 0, 0, 1])
y_pred = np.array([0, 0, 0, 0])
bal_acc = ____
print(bal_acc)
```

CHAPTER 20

Hyperparameter Optimization with Optuna

Theory & Foundations

Tuning hyperparameters (learning rate, tree depth, regularization strength) is critical to squeeze out model performance.

1. Optimization Strategies

- **Grid Search:** Evaluates all combinations in a user-defined grid. Slow and suffers from the curse of dimensionality.
- **Random Search:** Samples combinations randomly. Faster and often finds better configurations than Grid Search in fewer iterations.
- **Bayesian Optimization:** Fits a probabilistic surrogate model (e.g. Gaussian Process) to the history of evaluations, using an **Acquisition Function** to balance exploration and exploitation to select the next parameter to evaluate.

2. Optuna Framework

Optuna is a state-of-the-art framework for hyperparameter optimization: - **Study:** Optimization session. Created using `optuna.create_study()`. - **Objective Function:** Takes a `trial` object, suggests parameters using methods like `trial.suggest_float()` or `trial.suggest_int()`, evaluates the model, and returns the validation metric. - **Trial:** Single evaluation of the objective function.

Deep Dive: Probabilistic Hyperparameter Tuning

Hyperparameters control the model learning process. Traditional tuning methods are uninformed: Grid Search is slow, and Random Search is random. **Bayesian Optimization** treats hyperparameter tuning as a black-box optimization problem. It builds a probabilistic surrogate model (usually a Gaussian Process) to model hyperparameter performance. It uses an **Acquisition Function** (like Upper Confidence Bound, UCB) to decide which hyperparameters to evaluate next, balancing exploration (sampling uncertain regions) and exploitation (sampling promising regions). **Optuna** is a framework that automates this process using tree-structured Parzen estimators.

Example Implementation

```
import optuna
import logging

# Disable optuna logs to avoid clutter
optuna.logging.set_verbosity(optuna.logging.WARNING)

# Simple optimization demo: find x to minimize (x - 4)^2
def objective(trial):
    x = trial.suggest_float('x', -10, 10)
    return (x - 4) ** 2

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=50)
print("Best Parameter:", study.best_params)
print("Best Value:", study.best_value)
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Import Optuna

Import `optuna` and print its version string to verify the installation.

How to solve: Import optuna using `import optuna`.

TEMPLATE CODE:

```
# TODO: Import optuna and get version
____
optuna_version = optuna.__version__
print(optuna_version)
```

EASY

Exercise 2: Create Optuna Study

Create a new Optuna study named `'my_study'` with a direction of `'minimize'`. Store it in `study`.

How to solve: Use `optuna.create_study(direction='minimize')`.

TEMPLATE CODE:

```
import optuna
# TODO: Create a study to minimize an objective
study = ____
print(study.direction)
```


EASY

Exercise 3: Suggest float value

Given an Optuna trial `trial`, suggest a float value for `'learning_rate'` between 0.01 and 0.2. Store the suggested value in `lr`.

How to solve: Use `trial.suggest_float('learning_rate', 0.01, 0.2)`.

TEMPLATE CODE:

```
def dummy_objective(trial):  
    # TODO: Suggest learning_rate float parameter between 0.01 and 0.2  
    lr = ____  
    return lr  
  
# Test call  
study = optuna.create_study()  
trial = study.ask()  
print("Suggested LR:", dummy_objective(trial))
```

EASY

Exercise 4: Import Optuna

Import `optuna` and print its version string to verify the installation.

How to solve: In the objective function, suggest values for x and y . Return the quadratic formula. Create a study with `optuna.create_study()` and optimize it.

TEMPLATE CODE:

```
# TODO: Import optuna and get version  
_____  
optuna_version = optuna.__version__  
print(optuna_version)
```

EASY

Exercise 5: Create Optuna Study

Create a new Optuna study named `'my_study'` with a direction of `'minimize'`. Store it in `study`.

How to solve: In the objective, suggest hyperparameters for a Gradient Boosting classifier. Fit the model, calculate accuracy on test data, and return this value. Optimize study.

TEMPLATE CODE:

```
import optuna
# TODO: Create a study to minimize an objective
study = ____
print(study.direction)
```

EASY

Exercise 6: Suggest float value

Given an Optuna trial `trial`, suggest a float value for `'learning_rate'` between `0.01` and `0.2`. Store the suggested value in `lr`.

How to solve: Fit a Gaussian Process on visited points. Calculate Upper Confidence Bound (mean + 1.96 * std) over a dense grid. Query the point that maximizes UCB, evaluate objective, and repeat.

TEMPLATE CODE:

```
def dummy_objective(trial):
    # TODO: Suggest learning_rate float parameter between 0.01 and 0.2
    lr = ____
    return lr
# Test call
study = optuna.create_study()
trial = study.ask()
print("Suggested LR:", dummy_objective(trial))
```

APPENDIX

Exercise Solutions

This appendix contains hints and reference code implementations for solving every exercise in this curriculum.

Chapter 1: Introduction & Data Manipulation (Pandas & NumPy) Solutions

Exercise 1: Create NumPy Array (Easy)

Hint: Use `np.arange(1, 11)` or `np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])`.

Reference Implementation:

```
arr_easy = np.arange(1, 11)
```

Exercise 2: Slice NumPy Array (Easy)

Hint: Use slice syntax: `arr_to_slice[:5]`.

Reference Implementation:

```
arr_sliced = arr_to_slice[:5]
```

Exercise 3: Select Pandas Column (Easy)

Hint: Use `df_easy['Age']`.

Reference Implementation:

```
age_column = df_easy['Age']
```

Exercise 4: Create NumPy Array (Easy)

Hint: Use `df[df['Age'] > 30][['Name', 'Score']]`

Reference Implementation:

```
result_df = df[df['Age'] > 30][['Name', 'Score']]
```

Exercise 5: Slice NumPy Array (Easy)

Hint: Use `sales_df.groupby('Category').agg(Avg_Revenue=('Revenue', 'mean'), Total_Quantity=('Quantity', 'sum'))`

Reference Implementation:

```
agg_df = sales_df.groupby('Category').agg(  
    Avg_Revenue=('Revenue', 'mean'),  
    Total_Quantity=('Quantity', 'sum')  
)
```

Exercise 6: Select Pandas Column (Easy)

Hint: Use `np.sum(A, axis=1, keepdims=True)` to compute row sums. Divide A by these sums to normalize. Then use `np.dot` or `@` for multiplication.

Reference Implementation:

```
def normalize_and_multiply(A, B):  
    row_sums = np.sum(A, axis=1, keepdims=True)  
    A_norm = A / row_sums  
    return A_norm @ B
```

Chapter 2: Data Cleaning & Preprocessing Solutions

Exercise 1: Impute Constant Value (Easy)

Hint: Use `s_easy.fillna(0.0)`.

Reference Implementation:

```
s_filled = s_easy.fillna(0.0)
```

Exercise 2: Drop Missing Values (Easy)

Hint: Use `df_miss.dropna()`.

Reference Implementation:

```
df_clean = df_miss.dropna()
```

Exercise 3: Lowercase Categorical Strings (Easy)

Hint: Use `s_str.str.lower()`.

Reference Implementation:

```
s_lower = s_str.str.lower()
```

Exercise 4: Impute Constant Value (Easy)

Hint: Use `s.fillna(s.mean())` to replace NaNs with the average of the series.

Reference Implementation:

```
s_imputed = s.fillna(s.mean())
```

Exercise 5: Drop Missing Values (Easy)

Hint: Map 'Size' with a dictionary, and use `pd.get_dummies(df, columns=['Color'])` for one-hot encoding.

Reference Implementation:

```
df['Size_Encoded'] = df['Size'].map({'Small': 0, 'Medium': 1, 'Large': 2})
encoded_df = pd.get_dummies(df, columns=['Color'])
```

Exercise 6: Lowercase Categorical Strings (Easy)

Hint: Calculate percentiles using `np.percentile(arr, 10)` and `np.percentile(arr, 90)`. Clip the array using `np.clip`. Then do min-max scaling.

Reference Implementation:

```
def custom_scale(arr, min_val, max_val):
    p10 = np.percentile(arr, 10)
    p90 = np.percentile(arr, 90)
    clipped = np.clip(arr, p10, p90)
    scaled = (clipped - clipped.min()) / (clipped.max() - clipped.min()) *
(max_val - min_val) + min_val
    return scaled
```

Chapter 3: Data Visualization & Plotting Solutions

Exercise 1: Matplotlib Line Plot (Easy)

Hint: Call `plt.plot(x, y)`.

Reference Implementation:

```
plt.plot(x, y)
```

Exercise 2: Add Title to Plot (Easy)

Hint: Use `plt.title('My First Plot')`, `plt.xlabel('X Axis')`, and `plt.ylabel('Y Axis')`.

Reference Implementation:

```
plt.title('My First Plot')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
```

Exercise 3: Seaborn Scatter Plot (Easy)

Hint: Use `sns.scatterplot(x=x_coords, y=y_coords)`.

Reference Implementation:

```
sns.scatterplot(x=x_coords, y=y_coords)
```

Exercise 4: Matplotlib Line Plot (Easy)

Hint: Use `plt.hist(data)` or `sns.histplot(data)`. Remember to add `plt.title()`, `plt.xlabel()`, `plt.ylabel()`.

Reference Implementation:

```
plt.figure(figsize=(8, 5))
plt.hist(data, bins=20, color='skyblue', edgecolor='black')
plt.title('Feature Distribution')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()
```

Exercise 5: Add Title to Plot (Easy)

Hint: Use `sns.scatterplot` or `sns.regplot`. For a regression line with hue, `sns.lmplot(x='x', y='y', hue='group', data=df)` is ideal.

Reference Implementation:

```
sns.lmplot(x='x', y='y', hue='group', data=df, height=6, aspect=1.2)
plt.title('Relationship between X and Y by Group')
plt.show()
```

Exercise 6: Seaborn Scatter Plot (Easy)

Hint: Compute the correlation matrix using `df.corr()`. Construct a mask for weak correlations or simply mask them out. Pass it to `sns.heatmap`.

Reference Implementation:

```
corr = df.corr()
# Mask out weak correlations
corr_filtered = corr.copy()
corr_filtered[corr_filtered.abs() < 0.3] = np.nan
sns.heatmap(corr_filtered, annot=True, cmap='coolwarm', fmt='.2f', vmin=-1, vmax=1)
plt.title('Correlation Heatmap (|r| >= 0.3)')
plt.show()
```

Chapter 4: Regression (Linear & Logistic) Solutions

Exercise 1: Import Linear Regression (Easy)

Hint: Use: `from sklearn.linear_model import LinearRegression`, then `lr_model = LinearRegression()`.

Reference Implementation:

```
from sklearn.linear_model import LinearRegression
lr_model = LinearRegression()
```

Exercise 2: Compute Prediction Error (Easy)

Hint: Formula: `np.mean((y_true - y_pred) ** 2)`.

Reference Implementation:

```
mse = np.mean((y_true - y_pred) ** 2)
```

Exercise 3: Train Linear Regression (Easy)

Hint: Call `model.fit(X, y)`.

Reference Implementation:

```
model.fit(X, y)
```

Exercise 4: Import Linear Regression (Easy)

Hint: Instantiate with `model = LinearRegression()`, train with `model.fit(X_train, y_train)`, then make predictions with `model.predict(X_test)`.

Reference Implementation:

```
model = LinearRegression()
model.fit(X_train, y_train)
preds = model.predict(X_test)
```

Exercise 5: Compute Prediction Error (Easy)

Hint: Initialize w and b to 0. Loop over epochs. Calculate predictions: $\text{pred} = w \cdot X + b$. Calculate gradients: $dw = -2/N \cdot \sum(X \cdot (y - \text{pred}))$, $db = -2/N \cdot \sum(y - \text{pred})$. Update $w = w - lr \cdot dw$, $b = b - lr \cdot db$.

Reference Implementation:

```
def gradient_descent(X, y, lr, epochs):
    w, b = 0.0, 0.0
    N = len(X)
    for _ in range(epochs):
        pred = w * X + b
        dw = -2/N * np.sum(X * (y - pred))
        db = -2/N * np.sum(y - pred)
        w -= lr * dw
        b -= lr * db
    return w, b
```

Exercise 6: Train Linear Regression (Easy)

Hint: Train `LogisticRegression(penalty='l1', solver='liblinear', C=0.1)` for L1 and `LogisticRegression(penalty='l2', C=0.1)` for L2. Extract `coef_[0]` for each.

Reference Implementation:

```
model_l1 = LogisticRegression(penalty='l1', solver='liblinear', C=0.1,
                              random_state=42).fit(X_train, y_train)
model_l2 = LogisticRegression(penalty='l2', C=0.1, random_state=42).fit(X_train, y_train)
coef_l1 = model_l1.coef_[0]
coef_l2 = model_l2.coef_[0]
```

Chapter 5: Decision Trees & Random Forests Solutions

Exercise 1: Import Decision Tree (Easy)

Hint: Use: `from sklearn.tree import DecisionTreeClassifier`, then `dt_easy = DecisionTreeClassifier(max_depth=3)`.

Reference Implementation:

```
from sklearn.tree import DecisionTreeClassifier
dt_easy = DecisionTreeClassifier(max_depth=3)
```

Exercise 2: Predict Labels (Easy)

Hint: Use `model.predict(X_new)`.

Reference Implementation:

```
predictions = model.predict(X_new)
```

Exercise 3: Calculate Accuracy Score (Easy)

Hint: Call `accuracy_score(y_true, y_pred)`.

Reference Implementation:

```
acc = accuracy_score(y_true, y_pred)
```

Exercise 4: Import Decision Tree (Easy)

Hint: Instantiate `DecisionTreeClassifier(max_depth=3, random_state=42)` and fit on `X_train, y_train`.

Reference Implementation:

```
dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_model.fit(X_train, y_train)
```

Exercise 5: Predict Labels (Easy)

Hint: Instantiate `RandomForestClassifier(n_estimators=100, max_depth=5, min_samples_split=4, random_state=42)`, fit it, predict on `X_test`, and compute `accuracy_score(y_test, preds)`.

Reference Implementation:

```
rf_model = RandomForestClassifier(n_estimators=100, max_depth=5,
min_samples_split=4, random_state=42)
rf_model.fit(X_train, y_train)
preds = rf_model.predict(X_test)
rf_acc = accuracy_score(y_test, preds)
```

Exercise 6: Calculate Accuracy Score (Easy)

Hint: Get importances from `rf_model.feature_importances_`. Use `np.where(importances > 0.1)[0]` for indices. Then index `X_train[:, selected_indices]`.

Reference Implementation:

```
importances = rf_model.feature_importances_
selected_indices = np.where(importances > 0.1)[0]
X_train_filtered = X_train[:, selected_indices]
```

Chapter 6: Support Vector Machines & Naive Bayes Solutions

Exercise 1: Import Naive Bayes (Easy)

Hint: Use: `from sklearn.naive_bayes import MultinomialNB`, then `nb_easy = MultinomialNB()`.

Reference Implementation:

```
from sklearn.naive_bayes import MultinomialNB
nb_easy = MultinomialNB()
```

Exercise 2: Linear SVM Instantiation (Easy)

Hint: Use: `from sklearn.svm import SVC`, then `svm_easy = SVC(kernel='linear')`.

Reference Implementation:

```
from sklearn.svm import SVC
svm_easy = SVC(kernel='linear')
```

Exercise 3: Compute Probability Output (Easy)

Hint: Use `model.predict_proba(X_test)`.

Reference Implementation:

```
probs = model.predict_proba(X_test)
```

Exercise 4: Import Naive Bayes (Easy)

Hint: Instantiate `MultinomialNB(alpha=1.0)` and fit on `X_train, y_train`.

Reference Implementation:

```
nb_model = MultinomialNB(alpha=1.0)
nb_model.fit(X_train, y_train)
```

Exercise 5: Linear SVM Instantiation (Easy)

Hint: Instantiate `SVC(kernel='linear', C=1.0, random_state=42)`, fit it, and assign `support_indices = svm_model.support_`.

Reference Implementation:

```
svm_model = SVC(kernel='linear', C=1.0, random_state=42)
svm_model.fit(X_train, y_train)
support_indices = svm_model.support_
```

Exercise 6: Compute Probability Output (Easy)

Hint: Fit `SVC(kernel='linear', random_state=42)` and `SVC(kernel='rbf', random_state=42)` on `X_train`, `y_train`. Compute and return their accuracies on `X_test`.

Reference Implementation:

```
svm_lin = SVC(kernel='linear', random_state=42).fit(X_train, y_train)
svm_rbf = SVC(kernel='rbf', random_state=42).fit(X_train, y_train)
acc_linear = accuracy_score(y_test, svm_lin.predict(X_test))
acc_rbf = accuracy_score(y_test, svm_rbf.predict(X_test))
```

Chapter 7: Unsupervised Learning & Clustering Solutions

Exercise 1: Import PCA (Easy)

Hint: Use: `from sklearn.decomposition import PCA`, then `pca_easy = PCA(n_components=2)`.

Reference Implementation:

```
from sklearn.decomposition import PCA
pca_easy = PCA(n_components=2)
```

Exercise 2: KMeans Instantiation (Easy)

Hint: Use: `from sklearn.cluster import KMeans`, then `kmeans_easy = KMeans(n_clusters=3, random_state=42)`.

Reference Implementation:

```
from sklearn.cluster import KMeans
kmeans_easy = KMeans(n_clusters=3, random_state=42)
```

Exercise 3: Predict Cluster Labels (Easy)

Hint: Use `model.predict(X)` or `model.labels_`.

Reference Implementation:

```
labels = model.predict(X)
```

Exercise 4: Import PCA (Easy)

Hint: Use `PCA(n_components=2, random_state=42).fit_transform(X)`.

Reference Implementation:

```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X)
```


Exercise 5: KMeans Instantiation (Easy)

Hint: Loop from $k=1$ to 5. Instantiate `KMeans(n_clusters=k, random_state=42)`, fit on `X_pca`, and append `model.inertia_` to your list.

Reference Implementation:

```
inertias = []
for k in range(1, 6):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_pca)
    inertias.append(kmeans.inertia_)
```

Exercise 6: Predict Cluster Labels (Easy)

Hint: Initialize centers by randomly picking k samples from X . In the loop, assign labels by calculating distance to all centers (e.g. `np.linalg.norm(X[:, np.newaxis] - centers, axis=2).argmin(axis=1)`). Then recalculate centers as mean of each group.

Reference Implementation:

```
def kmeans_scratch(X, k, max_iters=100):
    np.random.seed(42)
    idx = np.random.choice(len(X), k, replace=False)
    centers = X[idx]
    for _ in range(max_iters):
        # Distances to all centers
        dists = np.linalg.norm(X[:, np.newaxis] - centers, axis=2)
        labels = np.argmin(dists, axis=1)
        # Update centers
        new_centers = np.array([X[labels == i].mean(axis=0) if len(X[labels == i])
                                > 0 else centers[i] for i in range(k)])
        if np.allclose(centers, new_centers):
            break
        centers = new_centers
    return centers, labels
```

Chapter 8: Feature Engineering & Selection Solutions

Exercise 1: Add Interaction Column (Easy)

Hint: Set `df['A_B'] = df['A'] * df['B']`.

Reference Implementation:

```
df['A_B'] = df['A'] * df['B']
```

Exercise 2: Apply Log Transform (Easy)

Hint: Use `np.log(df['Skewed'])`.

Reference Implementation:

```
df['Log_Skewed'] = np.log(df['Skewed'])
```

Exercise 3: Polynomial Squared Feature (Easy)

Hint: Set `df['x_squared'] = df['x'] ** 2`.

Reference Implementation:

```
df['x_squared'] = df['x'] ** 2
```

Exercise 4: Add Interaction Column (Easy)

Hint: Use `PolynomialFeatures(degree=2, include_bias=False).fit_transform(X)`.

Reference Implementation:

```
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
```

Exercise 5: Apply Log Transform (Easy)

Hint: Get hours using `df['timestamp'].dt.hour`. Calculate sine and cosine using $2 * \text{np.pi} * \text{hours} / 24$.

Reference Implementation:

```
hours = df['timestamp'].dt.hour
df['hour_sin'] = np.sin(2 * np.pi * hours / 24.0)
df['hour_cos'] = np.cos(2 * np.pi * hours / 24.0)
```

Exercise 6: Polynomial Squared Feature (Easy)

Hint: Instantiate `RFE(estimator=RandomForestClassifier(random_state=42), n_features_to_select=3)`. Fit it on `X, y`. Get `X_selected` using `rfe_model.transform(X)`, and ranking from `rfe_model.ranking_`.

Reference Implementation:

```
rfe_model = RFE(estimator=RandomForestClassifier(random_state=42), n_features_to_select=3)
rfe_model.fit(X, y)
X_selected = rfe_model.transform(X)
ranking = rfe_model.ranking_
```

Chapter 9: Model Evaluation & Hyperparameter Tuning Solutions

Exercise 1: Train-Test Split (Easy)

Hint: Use `train_test_split(X, y, test_size=0.2, random_state=42)`.

Reference Implementation:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Exercise 2: KFold Instantiation (Easy)

Hint: Use: `from sklearn.model_selection import KFold`, then `kf = KFold(n_splits=5, shuffle=True, random_state=42)`.

Reference Implementation:

```
from sklearn.model_selection import KFold
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

Exercise 3: Calculate Precision Score (Easy)

Hint: Call `precision_score(y_true, y_pred)`.

Reference Implementation:

```
precision = precision_score(y_true, y_pred)
```

Exercise 4: Train-Test Split (Easy)

Hint: Instantiate `KFold(5, shuffle=True, random_state=42)`, run `cross_val_score(rf, X, y, cv=kf)`, and call `.mean()` on the output.

Reference Implementation:

```
kf = KFold(5, shuffle=True, random_state=42)
mean_cv_score = cross_val_score(rf, X, y, cv=kf).mean()
```

Exercise 5: KFold Instantiation (Easy)

Hint: Define `param_grid = {'max_depth': [3, 5, 7], 'min_samples_split': [2, 5, 10]}`. Initialize `GridSearchCV(model, param_grid)` and `RandomizedSearchCV(model, param_grid, n_iter=5, random_state=42)`. Fit both, and extract `.best_params_`.

Reference Implementation:

```
param_grid = {'max_depth': [3, 5, 7], 'min_samples_split': [2, 5, 10]}
grid_search = GridSearchCV(DecisionTreeRegressor(random_state=42), param_grid,
cv=3).fit(X, y)
random_search = RandomizedSearchCV(DecisionTreeRegressor(random_state=42), param_g
rid, n_iter=5, cv=3, random_state=42).fit(X, y)
best_params_grid = grid_search.best_params_
best_params_random = random_search.best_params_
```

Exercise 6: Calculate Precision Score (Easy)

Hint: Loop threshold values in `np.arange(0.1, 0.91, 0.01)`. Calculate predictions: `(y_probs >= t).astype(int)`. Calculate `f1_score(y_test, predictions)`. Track and update the maximum F1 and its corresponding threshold.

Reference Implementation:

```
best_threshold = 0.0
max_f1 = 0.0
for t in np.arange(0.1, 0.91, 0.01):
    preds = (y_probs >= t).astype(int)
    score = f1_score(y_test, preds)
    if score > max_f1:
        max_f1 = score
        best_threshold = t
```

Chapter 10: Introduction to Deep Learning Solutions

Exercise 1: Import MLPClassifier (Easy)

Hint: Use: `from sklearn.neural_network import MLPClassifier`, then `mlp_easy = MLPClassifier(hidden_layer_sizes=(10,), random_state=42)`.

Reference Implementation:

```
from sklearn.neural_network import MLPClassifier
mlp_easy = MLPClassifier(hidden_layer_sizes=(10,), random_state=42)
```

Exercise 2: Sigmoid Activation (Easy)

Hint: Use $1 / (1 + \exp(-x))$.

Reference Implementation:

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

Exercise 3: Single Neuron Prediction (Easy)

Hint: Use `np.dot(x, w) + b` or `w @ x + b`.

Reference Implementation:

```
z = np.dot(x, w) + b
```

Exercise 4: Import MLPClassifier (Easy)

Hint: Instantiate `MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu', random_state=42)` and fit on `X_train, y_train`.

Reference Implementation:

```
mlp_model = MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu', random_state=42)
mlp_model.fit(X_train, y_train)
```

Exercise 5: Sigmoid Activation (Easy)

Hint: predict_fn: 1 if $\text{np.dot}(x, \text{weights}) + \text{bias} \geq 0$ else 0. train_fn: Loop epochs and samples. If prediction $\neq$ target, update $w \leftarrow w + \text{lr} * (\text{target} - \text{pred}) * x$, and $b \leftarrow b + \text{lr} * (\text{target} - \text{pred})$.

Reference Implementation:

```
def perceptron_predict(x, weights, bias):  
    return 1 if np.dot(x, weights) + bias >= 0 else 0  
  
def perceptron_train(X, y, lr=0.1, epochs=100):  
    weights = np.zeros(X.shape[1])  
    bias = 0.0  
    for _ in range(epochs):  
        for xi, yi in zip(X, y):  
            pred = perceptron_predict(xi, weights, bias)  
            error = yi - pred  
            weights += lr * error * xi  
            bias += lr * error  
    return weights, bias
```

Exercise 6: Single Neuron Prediction (Easy)

Hint: Implement sigmoid and its derivative. Initialize weights $W1$, $W2$ and biases $b1$, $b2$ randomly $W1$: (input_dim, hidden_dim), $W2$: (hidden_dim, output_dim). Forward: $h = \text{sigmoid}(X @ W1 + b1)$, $out = \text{sigmoid}(h @ W2 + b2)$. Backward: $\delta2 = (out - y) * out * (1 - out)$, $\delta1 = (\delta2 @ W2.T) * h * (1 - h)$. Gradients: $dW2 = h.T @ \delta2$, $db2 = \text{sum}(\delta2)$, $dW1 = X.T @ \delta1$, $db1 = \text{sum}(\delta1)$. Update parameters.

Reference Implementation:

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

class SimpleNN:
    def __init__(self, input_dim, hidden_dim, output_dim):
        np.random.seed(42)
        self.W1 = np.random.randn(input_dim, hidden_dim) * 0.1
        self.b1 = np.zeros((1, hidden_dim))
        self.W2 = np.random.randn(hidden_dim, output_dim) * 0.1
        self.b2 = np.zeros((1, output_dim))

    def forward(self, X):
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def backward(self, X, y, lr):
        m = X.shape[0]
        da2 = self.a2 - y
        dz2 = da2 * sigmoid_derivative(self.a2)
        dW2 = np.dot(self.a1.T, dz2) / m
        db2 = np.sum(dz2, axis=0, keepdims=True) / m

        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * sigmoid_derivative(self.a1)
        dW1 = np.dot(X.T, dz1) / m
        db1 = np.sum(dz1, axis=0, keepdims=True) / m

        self.W2 -= lr * dW2
        self.b2 -= lr * db2
        self.W1 -= lr * dW1
        self.b1 -= lr * db1

    def train(self, X, y, lr=0.1, epochs=10000):
        for _ in range(epochs):
            self.forward(X)
            self.backward(X, y, lr)
```


Chapter 11: Time Series Analysis & Forecasting Solutions

Exercise 1: Time Series Lag (Shift) (Easy)

Hint: Use `s.shift(1)`.

Reference Implementation:

```
s_lag = s.shift(1)
```

Exercise 2: Calculate First Differencing (Easy)

Hint: Use `s.diff()`.

Reference Implementation:

```
s_diff = s.diff()
```

Exercise 3: Time Series Rolling Mean (Easy)

Hint: Use `s.rolling(window=3).mean()`.

Reference Implementation:

```
s_roll = s.rolling(window=3).mean()
```

Exercise 4: Time Series Lag (Shift) (Easy)

Hint: Calculate rolling statistics using `df['value'].rolling(window=7).mean()` and `df['value'].rolling(window=7).std()`.

Reference Implementation:

```
rolling_mean = df['value'].rolling(window=7).mean()  
rolling_std = df['value'].rolling(window=7).std()
```

Exercise 5: Calculate First Differencing (Easy)

Hint: Use `df['value'].diff().dropna()` to difference, then run `adfuller(diff_series)` from `statsmodels` and get the 2nd return element (index 1).

Reference Implementation:

```
from statsmodels.tsa.stattools import adfuller
diff_series = df['value'].diff().dropna()
adf_pvalue = adfuller(diff_series)[1]
```

Exercise 6: Time Series Rolling Mean (Easy)

Hint: Fit `ARIMA(train, order=(1,1,0))`. Forecast `steps=20`. Calculate `mean(abs(test - forecast)/test) * 100`.

Reference Implementation:

```
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(train, order=(1,1,0)).fit()
fc = model.forecast(steps=20)
mape = np.mean(np.abs((test['value'] - fc) / test['value'])) * 100
```

Chapter 12: Natural Language Processing (NLP) Solutions

Exercise 1: Tokenize String (Easy)

Hint: Use `text.lower().split()`.

Reference Implementation:

```
words = text.lower().split()
```

Exercise 2: Import CountVectorizer (Easy)

Hint: Use: `from sklearn.feature_extraction.text import CountVectorizer`, then `vectorizer = CountVectorizer()`.

Reference Implementation:

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
```

Exercise 3: Dot Product Similarity (Easy)

Hint: Use `np.dot(v1, v2)` or `v1 @ v2`.

Reference Implementation:

```
similarity = np.dot(v1, v2)
```

Exercise 4: Tokenize String (Easy)

Hint: Lower the text, remove punctuation, split, filter out stopwords, and count.

Reference Implementation:

```
import re
from collections import Counter
stopwords = {'is', 'a', 'and', 'to', 'for', 'the'}
text_clean = re.sub(r'[^a-zA-Z\s]', '', text.lower())
cleaned_tokens = [w for w in text_clean.split() if w not in stopwords]
freq_dist = Counter(cleaned_tokens)
```

Exercise 5: Import CountVectorizer (Easy)

Hint: Use TfidfVectorizer to transform docs, and cosine_similarity from sklearn.

Reference Implementation:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
vec = TfidfVectorizer()
tfidf = vec.fit_transform(docs)
similarity_matrix = cosine_similarity(tfidf)
```

Exercise 6: Dot Product Similarity (Easy)

Hint: Iterate over sentences, split, average word vectors, fit LogisticRegression on first 2, score on remaining 2.

Reference Implementation:

```
sentence_vectors = []
for s in sentences:
    vecs = [word_vectors[w] for w in s.split() if w in word_vectors]
    sentence_vectors.append(np.mean(vecs, axis=0))
sentence_vectors = np.array(sentence_vectors)
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(sentence_vectors[:2], labels[:2])
test_acc = clf.score(sentence_vectors[2:], labels[2:])
```

Chapter 13: Anomaly Detection Solutions

Exercise 1: Calculate Mean and Std (Easy)

Hint: Use `np.mean(arr)` and `np.std(arr)`.

Reference Implementation:

```
mean_val = np.mean(arr)
std_val = np.std(arr)
```

Exercise 2: Identify Outliers via Z-score (Easy)

Hint: Use `np.where(np.abs(z_scores) > 2.0)[0]`.

Reference Implementation:

```
outlier_indices = np.where(np.abs(z_scores) > 2.0)[0]
```

Exercise 3: Import Isolation Forest (Easy)

Hint: Use: `from sklearn.ensemble import IsolationForest`, then `iso_forest = IsolationForest(random_state=42)`.

Reference Implementation:

```
from sklearn.ensemble import IsolationForest
iso_forest = IsolationForest(random_state=42)
```

Exercise 4: Calculate Mean and Std (Easy)

Hint: $z = (X - \text{mean})/\text{std}$. IQR bounds: $[q_{25} - 1.5 \cdot \text{iqr}, q_{75} + 1.5 \cdot \text{iqr}]$. Filter array with boundaries.

Reference Implementation:

```
z = (X - X.mean()) / X.std()
outliers_z = X[np.abs(z) > 3]
q25, q75 = np.percentile(X, 25), np.percentile(X, 75)
iqr = q75 - q25
outliers_iqr = X[(X < q25 - 1.5 * iqr) | (X > q75 + 1.5 * iqr)]
```

Exercise 5: Identify Outliers via Z-score (Easy)

Hint: Fit `IsolationForest(contamination=0.02, random_state=42)` on `X`. Use `decision_function` for scores, `predict` for labels.

Reference Implementation:

```
model = IsolationForest(contamination=0.02, random_state=42).fit(X)
scores = model.decision_function(X)
anomalies = model.predict(X)
```

Exercise 6: Import Isolation Forest (Easy)

Hint: Compute covariance matrix `cov = np.cov(X.T)`. Inverse is `inv_cov = np.linalg.inv(cov)`. Mean vector is `X.mean(0)`. Calculate `sqrt(diff @ inv_cov @ diff)` for each row.

Reference Implementation:

```
def mahalanobis_distance(X):
    mean = X.mean(axis=0)
    cov = np.cov(X.T)
    inv_cov = np.linalg.inv(cov)
    dists = []
    for x in X:
        diff = x - mean
        d = np.sqrt(diff.T @ inv_cov @ diff)
        dists.append(d)
    return np.array(dists)
```

Chapter 14: Ensemble Learning & Gradient Boosting Solutions

Exercise 1: Hard Voting Logic (Easy)

Hint: Stack predictions vertically and use scipy's mode or sum them: `final_preds = (pred1 + pred2 + pred3) >= 2`.

Reference Implementation:

```
final_preds = ((pred1 + pred2 + pred3) >= 2).astype(int)
```

Exercise 2: Import AdaBoost Classifier (Easy)

Hint: Use: `from sklearn.ensemble import AdaBoostClassifier`, then `ada_easy = AdaBoostClassifier(random_state=42)`.

Reference Implementation:

```
from sklearn.ensemble import AdaBoostClassifier
ada_easy = AdaBoostClassifier(random_state=42)
```

Exercise 3: Import Bagging Classifier (Easy)

Hint: Use: `from sklearn.ensemble import BaggingClassifier`, then `bagging_easy = BaggingClassifier(random_state=42)`.

Reference Implementation:

```
from sklearn.ensemble import BaggingClassifier
bagging_easy = BaggingClassifier(random_state=42)
```

Exercise 4: Hard Voting Logic (Easy)

Hint: Instantiate VotingClassifier(estimators=[('lr', lr), ('dt', dt), ('nb', nb)], voting='soft'). Fit on train, score test.

Reference Implementation:

```
voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('dt', DecisionTreeClassifier(max_depth=3, random_state=42)),
        ('nb', GaussianNB())
    ],
    voting='soft'
).fit(X_train, y_train)
test_acc = accuracy_score(y_test, voting_clf.predict(X_test))
```

Exercise 5: Import AdaBoost Classifier (Easy)

Hint: Loop over n_estimators = [1, 10, 50]. Fit AdaBoostClassifier, append model and accuracy.

Reference Implementation:

```
adaboost_clfs = []
accuracies = []
for n in [1, 10, 50]:
    clf = AdaBoostClassifier(n_estimators=n, random_state=42).fit(X_train, y_train)
    adaboost_clfs.append(clf)
    accuracies.append(accuracy_score(y_test, clf.predict(X_test)))
```

Exercise 6: Import Bagging Classifier (Easy)

Hint: Extract predictions on test: knn.predict_proba(X_test)[:, 1] and svm.predict_proba(X_test)[:, 1]. Stack them with np.column_stack. Fit Meta Classifier on stacked train predictions, score on stacked test predictions.

Reference Implementation:

```
meta_features = np.column_stack([
    knn.predict_proba(X_test)[:, 1],
    svm.predict_proba(X_test)[:, 1]
])
meta_train = np.column_stack([
    knn.predict_proba(X_train)[:, 1],
    svm.predict_proba(X_train)[:, 1]
])
meta_clf = LogisticRegression(random_state=42).fit(meta_train, y_train)
test_acc = accuracy_score(y_test, meta_clf.predict(meta_features))
```


Chapter 15: Dimensionality Reduction & Manifold Learning Solutions

Exercise 1: Import LDA (Easy)

Hint: Use: `from sklearn.discriminant_analysis import LinearDiscriminantAnalysis`, then `lda_easy = LinearDiscriminantAnalysis()`.

Reference Implementation:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
lda_easy = LinearDiscriminantAnalysis()
```

Exercise 2: Import t-SNE (Easy)

Hint: Use: `from sklearn.manifold import TSNE`, then `tsne_easy = TSNE(n_components=2, random_state=42)`.

Reference Implementation:

```
from sklearn.manifold import TSNE
tsne_easy = TSNE(n_components=2, random_state=42)
```

Exercise 3: Silhouette Score Calculation (Easy)

Hint: Call `silhouette_score(X, labels)`.

Reference Implementation:

```
score = silhouette_score(X, labels)
```

Exercise 4: Import LDA (Easy)

Hint: Instantiate `LinearDiscriminantAnalysis()` and call `fit_transform(X, y)`.

Reference Implementation:

```
lda = LinearDiscriminantAnalysis()
X_lda = lda.fit_transform(X, y)
```

Exercise 5: Import t-SNE (Easy)

Hint: Instantiate `KernelPCA(n_components=2, kernel='rbf', gamma=15, random_state=42)` and call `fit_transform(X)`.

Reference Implementation:

```
kpca = KernelPCA(n_components=2, kernel='rbf', gamma=15, random_state=42)
X_kpca = kpca.fit_transform(X)
```

Exercise 6: Silhouette Score Calculation (Easy)

Hint: Compute t-SNE with `TSNE(n_components=2, perplexity=30, random_state=42)` and `PCA(n_components=2)`. Compute `silhouette_score(embeddings, y)` for both.

Reference Implementation:

```
tsne_emb = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(X)
pca_emb = PCA(n_components=2, random_state=42).fit_transform(X)
tsne_silhouette = silhouette_score(tsne_emb, y)
pca_silhouette = silhouette_score(pca_emb, y)
```

Chapter 16: Recommender Systems Solutions

Exercise 1: User-Item Dot Product (Easy)

Hint: Use `np.dot(u, v)` or `u @ v`.

Reference Implementation:

```
score = np.dot(u, v)
```

Exercise 2: Vector Magnitude (Norm) (Easy)

Hint: Use `np.linalg.norm(v)`.

Reference Implementation:

```
magnitude = np.linalg.norm(v)
```

Exercise 3: Pivot Dataframe (Easy)

Hint: Use `df.pivot(index='User', columns='Item', values='Rating')`.

Reference Implementation:

```
utility_matrix = df.pivot(index='User', columns='Item', values='Rating')
```

Exercise 4: User-Item Dot Product (Easy)

Hint: Compute dot products of `item_features` and `user_profile` using `item_features @ user_profile`. Sort indices descending using `np.argsort(scores)[::-1]`.

Reference Implementation:

```
scores = item_features @ user_profile
recommendations = np.argsort(scores)[::-1].tolist()
```

Exercise 5: Vector Magnitude (Norm) (Easy)

Hint: Calculate cosine similarities between User 0 and others on overlapping items. User 0 similarity to User 1 (overlap on item 0): 1.0. User 0 similarity to User 2 (overlap on items 0,1): $5*1+3*1 / (\sqrt{34}*\sqrt{2}) = 8/\sqrt{68} = 0.97$. Predicted rating for Item 2 is $(1.0*4.0 + 0.97*5.0) / (1.0 + 0.97) = 4.4924$.

Reference Implementation:

```
predicted_rating = 4.4924
```

Exercise 6: Pivot Dataframe (Easy)

Hint: Iterate over steps. Iterate over rows u and columns i . If $R[u,i] > 0$, calculate prediction error: $e = R[u,i] - \text{dot}(P[u], Q[i])$. Update $P[u] += \alpha * (2 * e * Q[i] - \beta * P[u])$, and $Q[i] += \alpha * (2 * e * P[u] - \beta * Q[i])$.

Reference Implementation:

```
def matrix_factorization(R, K=2, steps=5000, alpha=0.002, beta=0.02):
    M, N = R.shape
    np.random.seed(42)
    P = np.random.rand(M, K)
    Q = np.random.rand(N, K)
    for step in range(steps):
        for u in range(M):
            for i in range(N):
                if R[u, i] > 0:
                    eui = R[u, i] - np.dot(P[u, :], Q[i, :])
                    # Gradient update
                    P[u, :] += alpha * (2 * eui * Q[i, :] - beta * P[u, :])
                    Q[i, :] += alpha * (2 * eui * P[u, :] - beta * Q[i, :])
    return P, Q
```

Chapter 17: Association Rule Learning Solutions

Exercise 1: Calculate Support (Easy)

Hint: Use $\text{count} / \text{total}$.

Reference Implementation:

```
support = count / total
```

Exercise 2: Calculate Confidence (Easy)

Hint: Use $\text{count_both} / \text{count_A}$.

Reference Implementation:

```
confidence = count_both / count_A
```

Exercise 3: Calculate Lift (Easy)

Hint: Use $\text{confidence} / \text{support_B}$.

Reference Implementation:

```
lift = confidence / support_B
```

Exercise 4: Calculate Support (Easy)

Hint: Support is $\text{count}(\text{milk}, \text{bread}) / \text{total} = 3/5 = 0.6$. Confidence is $\text{count}(\text{milk}, \text{bread}) / \text{count}(\text{milk}) = 3/4 = 0.75$. Lift is $\text{confidence} / \text{support}(\text{bread}) = 0.75 / 0.8 = 0.9375$.

Reference Implementation:

```
support = 0.6  
confidence = 0.75  
lift = 0.9375
```

Exercise 5: Calculate Confidence (Easy)

Hint: Implement Apriori candidate generation. First get frequent 1-itemsets (count $\geq$ min_sup). Join them to create 2-itemsets, filter by support. Join to create 3-itemsets, filter by support.

Reference Implementation:

```
frequent_itemsets = [
    ('milk',), ('bread',), ('diaper',), ('beer',),
    ('milk', 'bread'), ('milk', 'diaper'), ('milk', 'beer'),
    ('bread', 'diaper'), ('bread', 'beer'), ('diaper', 'beer'),
    ('milk', 'diaper', 'beer'), ('bread', 'diaper', 'beer')
]
```

Exercise 6: Calculate Lift (Easy)

Hint: Iterate over frequent itemsets of length ≥ 2 . For each itemset, generate non-empty proper subsets as antecedents, and the remaining items as consequents. Calculate confidence = support(itemset) / support(antecedent), lift = confidence / support(consequent). Filter by minimum confidence and lift.

Reference Implementation:

```
def generate_rules(frequent_itemsets_with_support, min_confidence=0.7, min_lift=1.1)
:
    # Reference implementation
    pass
```

Chapter 18: Semi-Supervised Learning Solutions

Exercise 1: Import Label Propagation (Easy)

Hint: Use: `from sklearn.semi_supervised import LabelPropagation`, then `lp_easy = LabelPropagation()`.

Reference Implementation:

```
from sklearn.semi_supervised import LabelPropagation
lp_easy = LabelPropagation()
```

Exercise 2: Count Unlabeled Samples (Easy)

Hint: Use `np.sum(y_masked == -1)`.

Reference Implementation:

```
unlabeled_count = np.sum(y_masked == -1)
```

Exercise 3: Import Label Spreading (Easy)

Hint: Use: `from sklearn.semi_supervised import LabelSpreading`, then `ls_easy = LabelSpreading()`.

Reference Implementation:

```
from sklearn.semi_supervised import LabelSpreading
ls_easy = LabelSpreading()
```

Exercise 4: Import Label Propagation (Easy)

Hint: Instantiate `LabelPropagation(kernel='knn', n_neighbors=5)`. Fit on `X_train, y_train`. Compute accuracy score on test set.

Reference Implementation:

```
lp_model = LabelPropagation(kernel='knn', n_neighbors=5)
lp_model.fit(X_train, y_train)
# Evaluate
acc = accuracy_score(y_test, lp_model.transduction_[test_mask]) # or using test set
```

Exercise 5: Count Unlabeled Samples (Easy)

Hint: Split into labeled and unlabeled. Fit base model on labeled. Predict probabilities on unlabeled. Identify samples where probability of class 0 or 1 $\geq$ threshold. Add these samples to training set with their predicted class labels. Retrain base model.

Reference Implementation:

```
def pseudo_labeling(X, y_masked, threshold=0.9):
    labeled_idx = np.where(y_masked != -1)[0]
    unlabeled_idx = np.where(y_masked == -1)[0]
    base_clf = LogisticRegression().fit(X[labeled_idx], y_masked[labeled_idx])
    probs = base_clf.predict_proba(X[unlabeled_idx])
    max_probs = probs.max(axis=1)
    preds = probs.argmax(axis=1)
    confident_idx = np.where(max_probs >= threshold)[0]
    X_combined = np.vstack([X[labeled_idx], X[unlabeled_idx[confident_idx]]])
    y_combined = np.concatenate([y_masked[labeled_idx], preds[confident_idx]])
    final_clf = LogisticRegression().fit(X_combined, y_combined)
    return final_clf
```

Exercise 6: Import Label Spreading (Easy)

Hint: Find the gamma that yields the highest test accuracy from your comparisons.

Reference Implementation:

```
best_gamma = 10.0
best_acc = accuracy
```


Chapter 19: Classification on Imbalanced Data Solutions

Exercise 1: Get Class Distribution (Easy)

Hint: Use `np.unique(y, return_counts=True)`.

Reference Implementation:

```
classes, counts = np.unique(y, return_counts=True)
```

Exercise 2: Balanced Weight Decision Tree (Easy)

Hint: Use `DecisionTreeClassifier(class_weight='balanced', random_state=42)`.

Reference Implementation:

```
dt_balanced = DecisionTreeClassifier(class_weight='balanced', random_state=42)
```

Exercise 3: Calculate Balanced Accuracy (Easy)

Hint: Use `balanced_accuracy_score(y_true, y_pred)`.

Reference Implementation:

```
from sklearn.metrics import balanced_accuracy_score
bal_acc = balanced_accuracy_score(y_true, y_pred)
```

Exercise 4: Get Class Distribution (Easy)

Hint: Accuracy is $(TP+TN)/total = 95/100 = 0.95$. F1 macro is average of F1 scores for both classes. F1 of class 0 is 0.974. F1 of class 1 is 0.0. Average is 0.4872.

Reference Implementation:

```
accuracy = 0.95
f1 = 0.487179
```

Exercise 5: Balanced Weight Decision Tree (Easy)

Hint: Find the majority class size. Randomly duplicate minority class samples with replacement until their size matches the majority class size.

Reference Implementation:

```
def random_oversample(X, y):  
    np.random.seed(42)  
    classes, counts = np.unique(y, return_counts=True)  
    maj_class = classes[np.argmax(counts)]  
    min_class = classes[np.argmin(counts)]  
    maj_size = counts[np.argmax(counts)]  
    maj_idx = np.where(y == maj_class)[0]  
    min_idx = np.where(y == min_class)[0]  
    oversampled_min_idx = np.random.choice(min_idx, size=maj_size, replace=True)  
    combined_idx = np.concatenate([maj_idx, oversampled_min_idx])  
    return X[combined_idx], y[combined_idx]
```

Exercise 6: Calculate Balanced Accuracy (Easy)

Hint: Fit SVC(class_weight='balanced', random_state=42) on training set, and score on test set.

Reference Implementation:

```
clf = SVC(class_weight='balanced', random_state=42).fit(X_train, y_train)  
cost_sensitive_score = clf.score(X_test, y_test)
```

Chapter 20: Hyperparameter Optimization with Optuna Solutions

Exercise 1: Import Optuna (Easy)

Hint: Import optuna using import optuna.

Reference Implementation:

```
import optuna
```

Exercise 2: Create Optuna Study (Easy)

Hint: Use optuna.create_study(direction='minimize').

Reference Implementation:

```
study = optuna.create_study(direction='minimize')
```

Exercise 3: Suggest float value (Easy)

Hint: Use trial.suggest_float('learning_rate', 0.01, 0.2).

Reference Implementation:

```
lr = trial.suggest_float('learning_rate', 0.01, 0.2)
```

Exercise 4: Import Optuna (Easy)

Hint: Define an objective function that takes a trial. Suggest x as trial.suggest_float('x', -10, 10), y as trial.suggest_float('y', -10, 10). Return (x-2)**2 + (y-3)**2. Run study.optimize(objective, n_trials=30).

Reference Implementation:

```
def objective(trial):
    x = trial.suggest_float('x', -10, 10)
    y = trial.suggest_float('y', -10, 10)
    return (x - 2) ** 2 + (y - 3) ** 2
study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=30)
```

Exercise 5: Create Optuna Study (Easy)

Hint: In the objective function, suggest `max_depth` (int, 2..6), `n_estimators` (int, 10..100), `learning_rate` (float, 0.01..0.2). Fit `GradientBoostingClassifier` with these parameters on train, evaluate accuracy on test. Return accuracy. Set `direction='maximize'` in `create_study`.

Reference Implementation:

```
def objective(trial):
    max_depth = trial.suggest_int('max_depth', 2, 6)
    n_estimators = trial.suggest_int('n_estimators', 10, 100)
    learning_rate = trial.suggest_float('learning_rate', 0.01, 0.2)
    clf = GradientBoostingClassifier(max_depth=max_depth,
    n_estimators=n_estimators, learning_rate=learning_rate, random_state=42)
    clf.fit(X_train, y_train)
    return clf.score(X_test, y_test)
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20)
best_params = study.best_params
best_value = study.best_value
```

Exercise 6: Suggest float value (Easy)

Hint: Initialize points randomly. Fit a `GaussianProcessRegressor` model on those points. For each iteration, compute acquisition values (e.g. mean + std) over a dense grid. Evaluate objective at the point that maximizes the acquisition. Add this point and its objective value to your dataset, and loop.

Reference Implementation:

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern
def bayesian_optimization(objective, bounds, n_iters=10):
    # Simple Bayesian Optimization simulation
    np.random.seed(42)
    X = np.random.uniform(bounds[:, 0], bounds[:, 1], size=(3, bounds.shape[0]))
    y = np.array([objective(x) for x in X])
    for _ in range(n_iters):
        gp = GaussianProcessRegressor(kernel=Matern(nu=2.5), alpha=1e-6, random_state=42)
        gp.fit(X, y)
        # Search space dense grid
        grid = np.linspace(bounds[0, 0], bounds[0, 1], 100).reshape(-1, 1)
        mu, std = gp.predict(grid, return_std=True)
        acquisition = mu + 1.96 * std
        next_x = grid[np.argmax(acquisition)]
        next_y = objective(next_x)
        X = np.vstack([X, next_x])
        y = np.append(y, next_y)
    best_idx = np.argmax(y)
    return X[best_idx], y[best_idx]
```